



Advanced Machine Learning 26–1

Scalable Diffusion Models with Transformers

Team 3 – Valerie Kinson & Changhyeon Leem

- DiT reproducing & Hypothesis evaluation project

Contents

- | | | | |
|---|------------------------------|----|-------------------|
| 1 | DiT – Background | 6 | HiT – Preliminary |
| 2 | DiT – Method | 7 | HiT – Method |
| 3 | DiT – Experiment | 8 | HiT – Experiment |
| 4 | DiT – Conclusion | 9 | HiT – Conclusion |
| 5 | HiT – Intuition & Hypothesis | 10 | HiT – Discussion |

DiT

1 DiT – Background

Before DiT, Diffusion Backbone was U-Net.

U-Net has **architecture level** inductive bias.

So, we must use a fixed network shape.

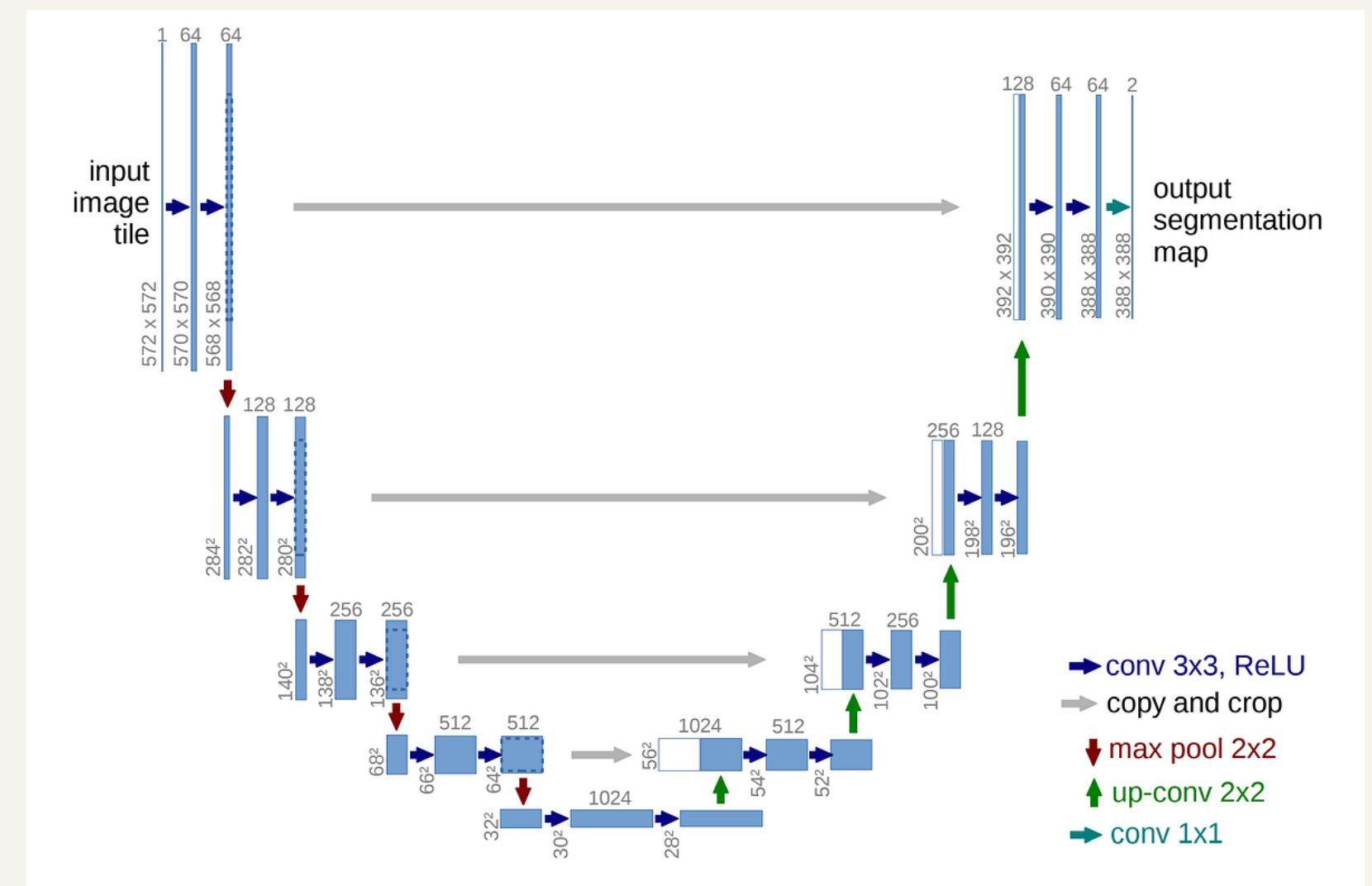


Figure 1. The U-Net architecture, the standard diffusion backbone before DiT. Multi-scale structure is built into the network. Adapted from Ronneberger, O., Fischer, P., & Brox, T. (2015). U-Net: Convolutional networks for biomedical image segmentation. In *Medical Image Computing and Computer-Assisted Intervention (MICCAI)* (pp. 234-241). Springer.

1 DiT – Background

The transformer model has **layer level** inductive bias.

That makes model design more flexible.

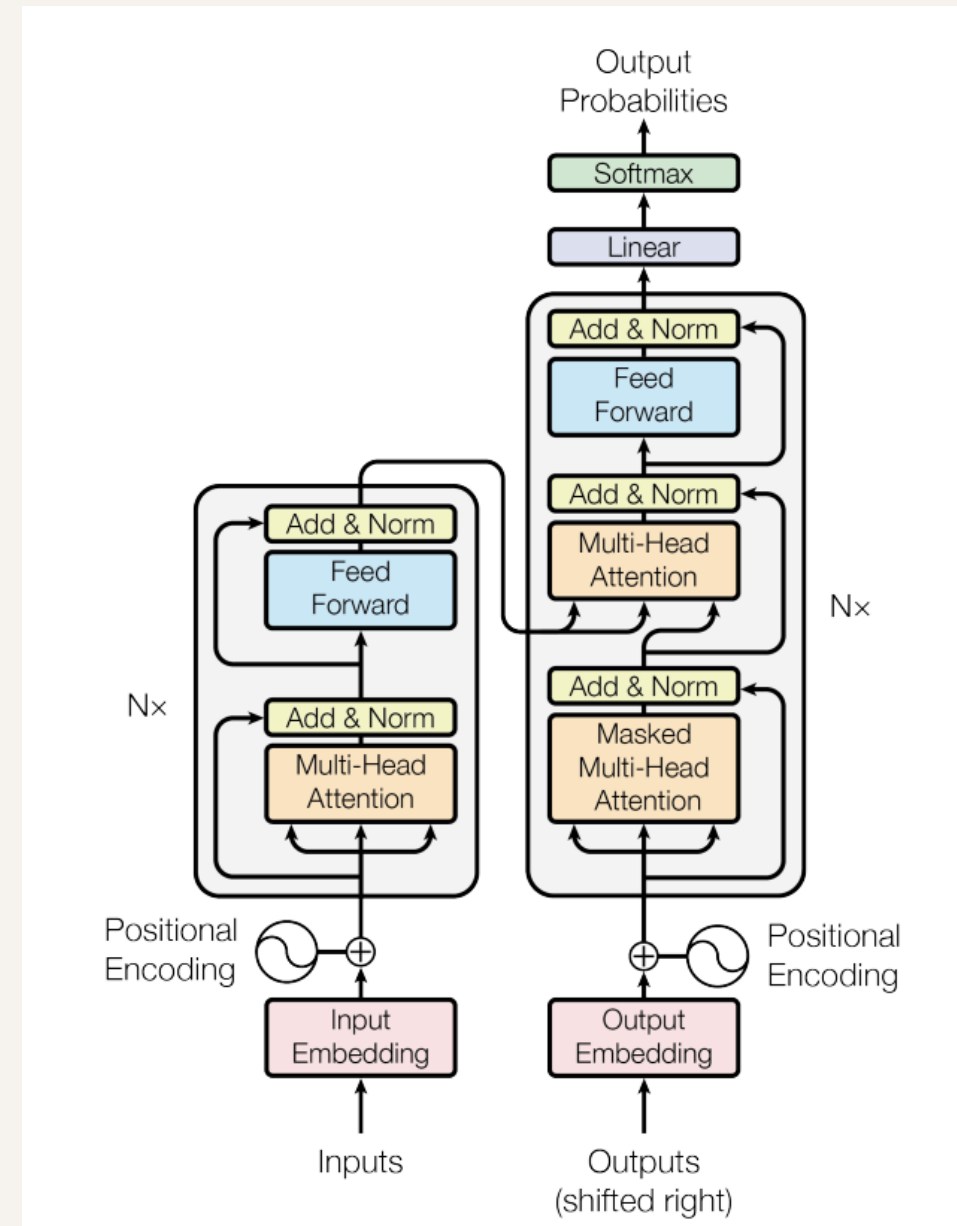


Figure 2. The Transformer architecture, where the inductive bias sits at the layer level. Adapted from Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., Kaiser, Ł., & Polosukhin, I. (2017). Attention is all you need. In *Advances in Neural Information Processing Systems (NeurIPS)* (Vol. 30, pp. 5998-6008).

1 DiT – Background

The adaptability of transformer models across many domains had been proven by prior research such as ViT.

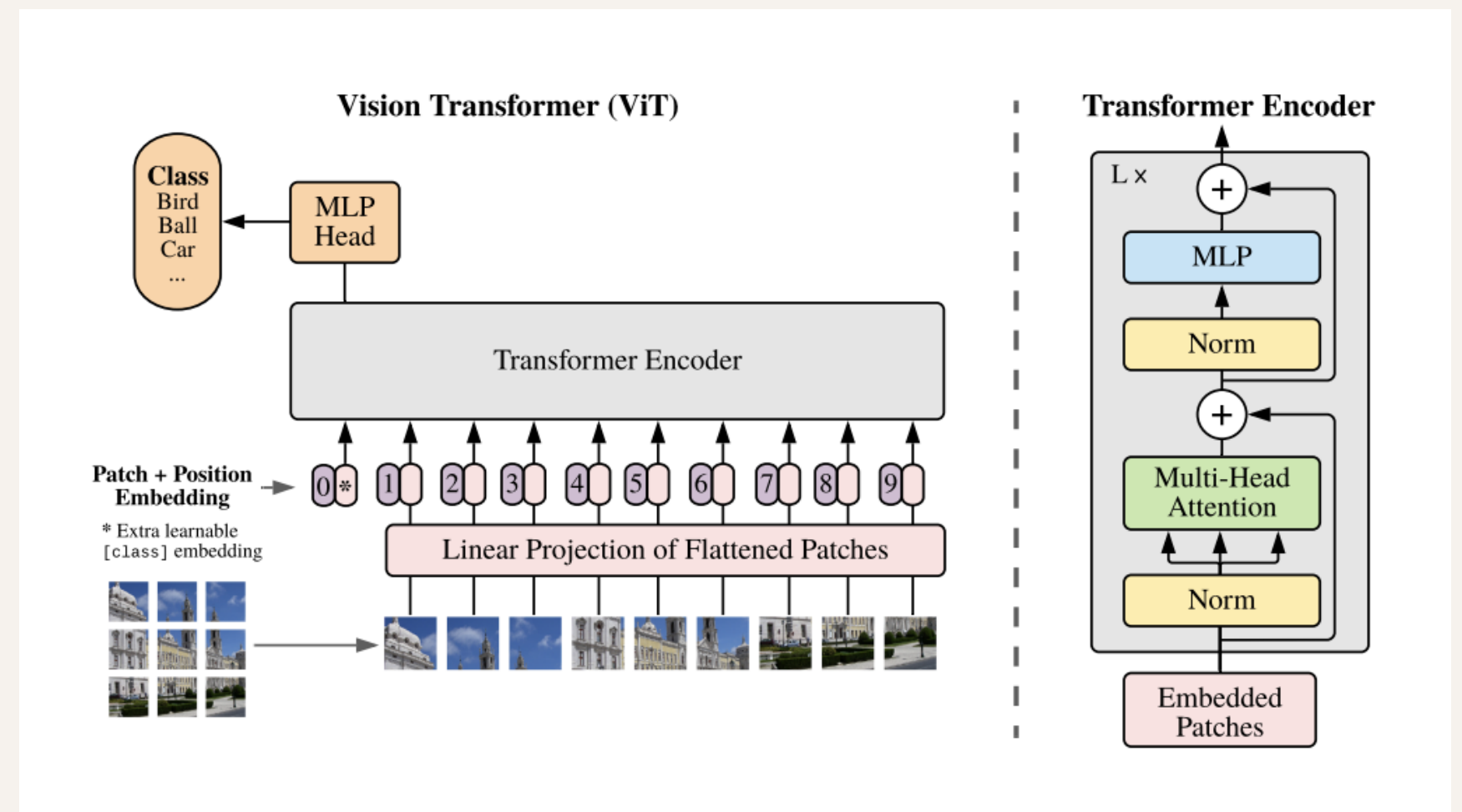


Figure 3. Vision Transformer (ViT), which showed a scaling law for Transformers in vision. Adapted from Dosovitskiy, A., et al. (2021). An image is worth 16x16 words: Transformers for image recognition at scale. In International Conference on Learning Representations (ICLR).

2 DiT – Method

- > configs
- > data
- > diffusion
- > evaluation
- > fid_check
- > frozen_remeasure
- > frozen_results
- > interpolation_results
- > models
- > paper
- > scripts
- > training

2 DiT – Method

DiT Model trains on VAE latent.

We choose AdaLN as conditioning option, which was introduced in the original paper.

$$L_{\epsilon-MSE} = \mathbb{E}_{t, x_0, \epsilon} [\|\epsilon - \epsilon_{\theta}(x_t, t)\|^2]$$

$$L_{vb} = \mathbb{E}_{t, x_0, \epsilon} [D_{KL}(q(x_{t-1}|x_t, x_0) \parallel p_{\theta}(x_{t-1}|x_t))]$$

$$L_{total} = L_{\epsilon-MSE} + \lambda L_{vb}$$

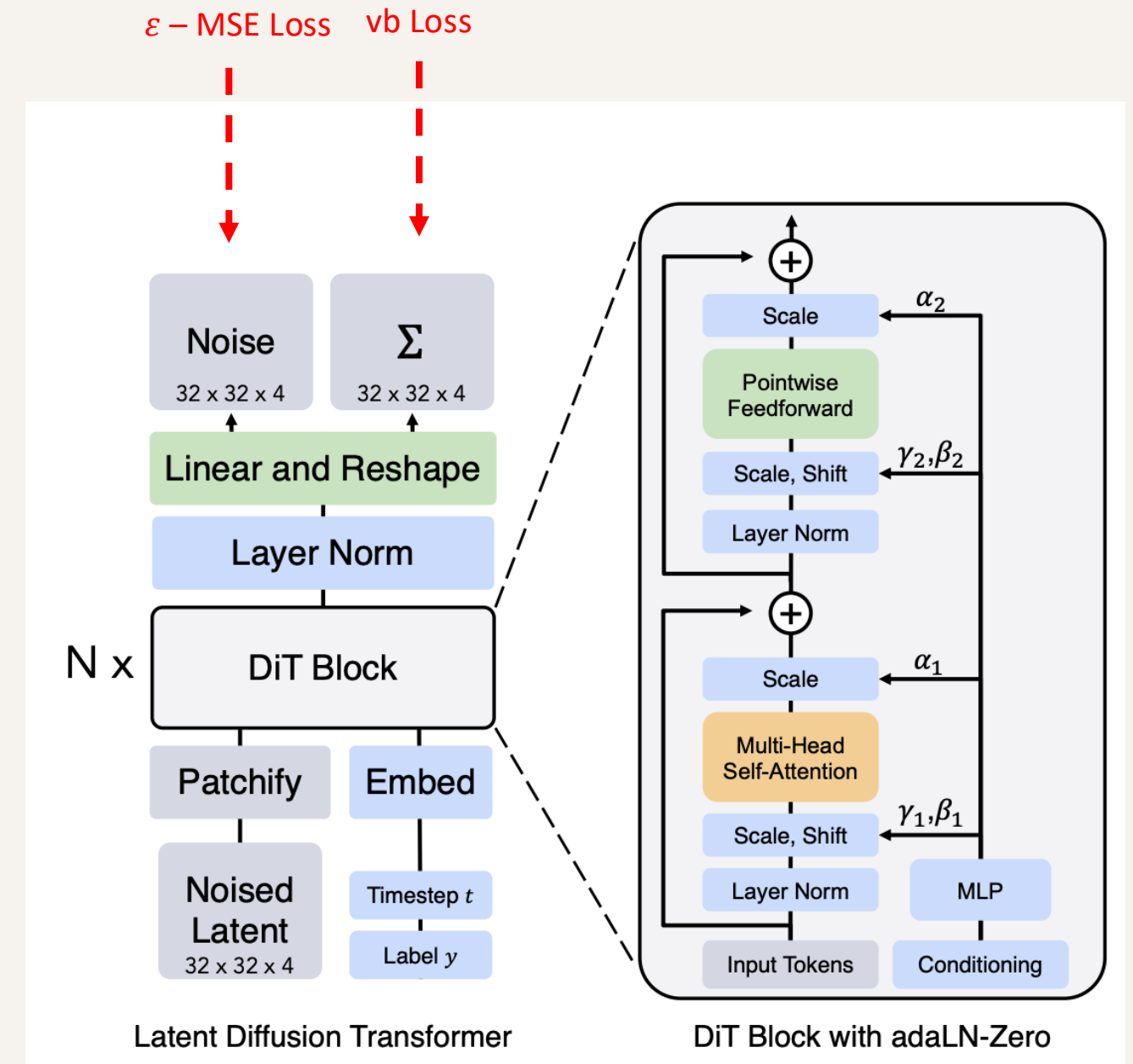


Figure 4. The DiT architecture: a latent diffusion model with a Transformer backbone and an adaLN-Zero block for conditioning. Adapted from Peebles, W., & Xie, S. (2023). Scalable diffusion models with transformers. In *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)* (pp. 4195-4205)..

2 DiT – Method

a. Code of Loading VAE latents

```
18 def download_imagenet_latents(output_dir, subset_classes=None, max_samples=None):
25     ds = load_dataset("Forbu14/imagenet-1k-latent", split="train", streaming=True)
26
27     os.makedirs(output_dir, exist_ok=True)
28     class_counters = {}
29     total_saved = 0
```

```
9 class LatentDataset(Dataset):
47     def __getitem__(self, idx):
48         fpath, label = self.samples[idx]
49         data = torch.load(fpath, map_location='cpu', weights_only=True)
50
51         if isinstance(data, dict):
52             latent = data['latent']
53         else:
54             latent = data
55
56         latent = latent.float()
57
58         if self.transform is not None:
59             latent = self.transform(latent)
60
61         return latent, label
```

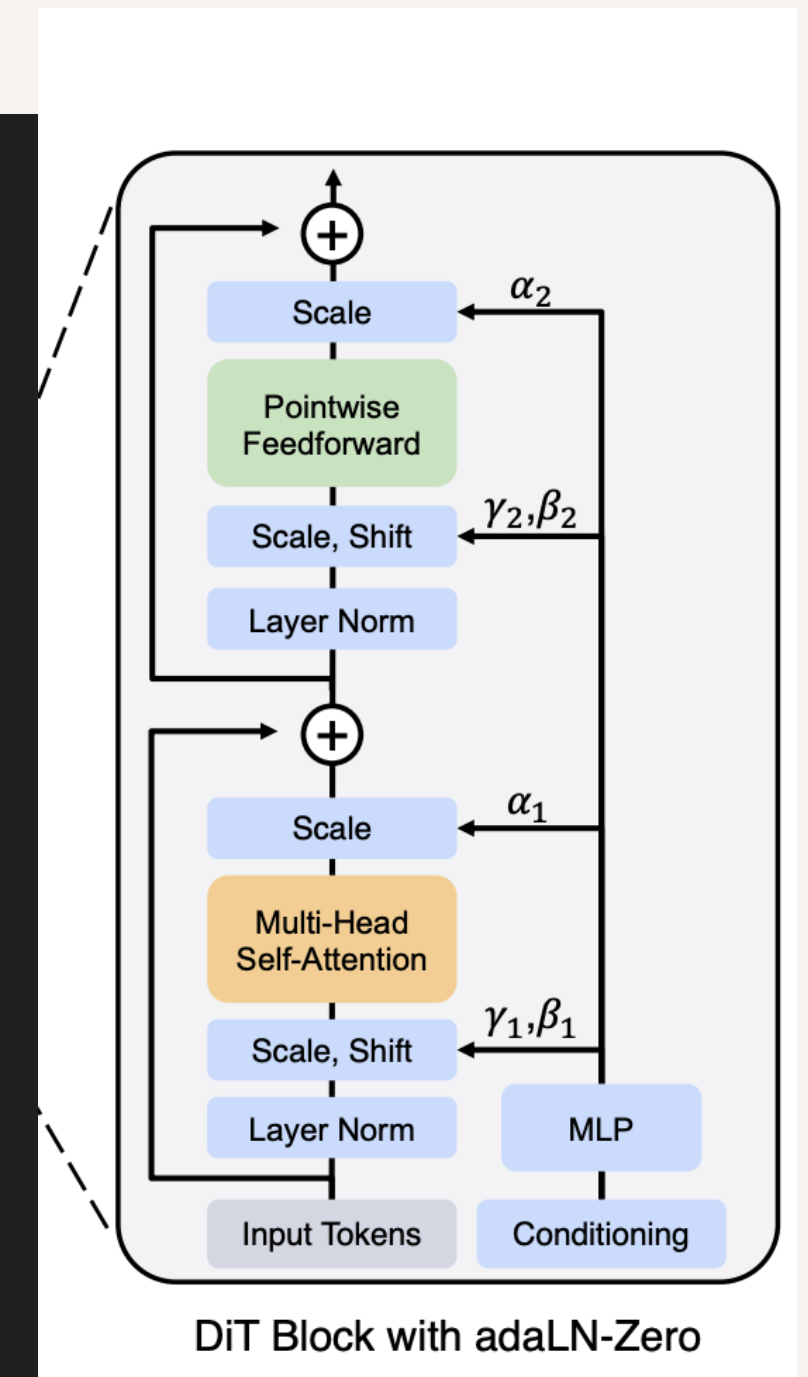
```
if vae is not None:
    with torch.no_grad():
        samples = vae.decode(samples.float() / 0.18215).sample
```

2 DiT – Method

b. Code of Loading DiT Block

```
40 class Attention(nn.Module):
41     """Multi-head self-attention."""
42
43     def __init__(
44         self,
45         dim: int,
46         num_heads: int = 8,
47         qkv_bias: bool = True,
48     ):
49         super().__init__()
50         self.num_heads = num_heads
51         self.head_dim = dim // num_heads
52         self.qkv = nn.Linear(dim, dim * 3, bias=qkv_bias)
53         self.proj = nn.Linear(dim, dim)
54
55     def forward(self, x: torch.Tensor) -> torch.Tensor:
56         B, N, C = x.shape
57         qkv = self.qkv(x).reshape(B, N, 3, self.num_heads, self.head_dim)
58         qkv = qkv.permute(2, 0, 3, 1, 4)
59         q, k, v = qkv.unbind(0)
60         # scaled dot-product (PyTorch 2.0+)
61         x = F.scaled_dot_product_attention(q, k, v)
62         x = x.transpose(1, 2).reshape(B, N, C)
63         return self.proj(x)
```

```
183 class DiTBlock(nn.Module):
184     """adaLN-Zero transformer block.
185
186     Predicts 6 modulation params (shift/scale/gate) from conditioning (t+y),
187     applied to attention+MLP. Zero-initialized so blocks start as identity.
188     """
189
190     def __init__(
191         self, hidden_size: int, num_heads: int, mlp_ratio: float = 4.0, **kwargs
192     ):
193         super().__init__()
194         self.norm1 = nn.LayerNorm(hidden_size, elementwise_affine=False, eps=1e-6)
195         self.attn = Attention(hidden_size, num_heads=num_heads, qkv_bias=True)
196         self.norm2 = nn.LayerNorm(hidden_size, elementwise_affine=False, eps=1e-6)
197         self.mlp = Mlp(
198             in_features=hidden_size,
199             hidden_features=int(hidden_size * mlp_ratio),
200         )
201         # (shift, scale, gate) x (attn, mlp) = 6 params
202         self.adaLN_modulation = nn.Sequential(
203             nn.SiLU(),
204             nn.Linear(hidden_size, 6 * hidden_size, bias=True),
205         )
206
207     def forward(self, x: torch.Tensor, c: torch.Tensor) -> torch.Tensor:
208         shift_msa, scale_msa, gate_msa, shift_mlp, scale_mlp, gate_mlp = (
209             self.adaLN_modulation(c).chunk(6, dim=1)
210         )
211         x = x + gate_msa.unsqueeze(1) * self.attn(
212             modulate(self.norm1(x), shift_msa, scale_msa)
213         )
214         x = x + gate_mlp.unsqueeze(1) * self.mlp(
215             modulate(self.norm2(x), shift_mlp, scale_mlp)
216         )
217         return x
```



$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

$$\text{head}_i = \text{Attention}(QW_i^Q, KW_i^K, VW_i^V)$$

$$\text{MultiHead}(Q, K, V) = \text{Concat}(\text{head}_1, \dots, \text{head}_h)W^O$$

2 DiT – Method

b. Code of Loading DiT Block

```
88 def _sincos_1d(embed_dim: int, pos: np.ndarray) -> np.ndarray:
89     """1D sincos positional embedding."""
90     omega = np.arange(embed_dim // 2, dtype=np.float64)
91     omega /= embed_dim / 2.0
92     omega = 1.0 / (10000 ** omega)
93     out = np.einsum("m,d->md", pos.ravel(), omega)
94     return np.concatenate([np.sin(out), np.cos(out)], axis=1)
95
96
97 def sincos_2d_pos_embed(embed_dim: int, grid_size: int) -> np.ndarray:
98     """2D sincos positional embedding (square grid)."""
99     gh = np.arange(grid_size, dtype=np.float32)
100     gw = np.arange(grid_size, dtype=np.float32)
101     grid = np.stack(np.meshgrid(gw, gh), axis=0).reshape(2, 1, grid_size, grid_size)
102     h_emb = _sincos_1d(embed_dim // 2, grid[0])
103     w_emb = _sincos_1d(embed_dim // 2, grid[1])
104     return np.concatenate([h_emb, w_emb], axis=1)
```

2 DiT – Method

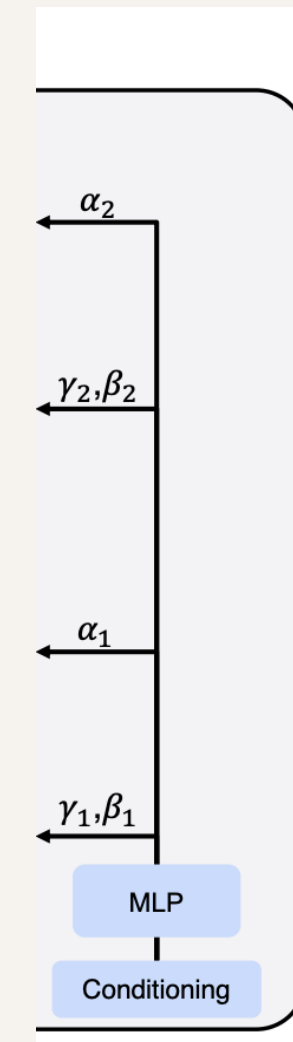
c. Code of adaLN

```
201 # (shift, scale, gate) x (attn, mlp) = 6 params
202 self.adaLN_modulation = nn.Sequential(
203     nn.SiLU(),
204     nn.Linear(hidden_size, 6 * hidden_size, bias=True),
```

```
79 def modulate(x: torch.Tensor, shift: torch.Tensor, scale: torch.Tensor) -> torch.Tensor:
80     """adaLN modulation: x * (1 + scale) + shift."""
81     return x * (1 + scale.unsqueeze(1)) + shift.unsqueeze(1)
```

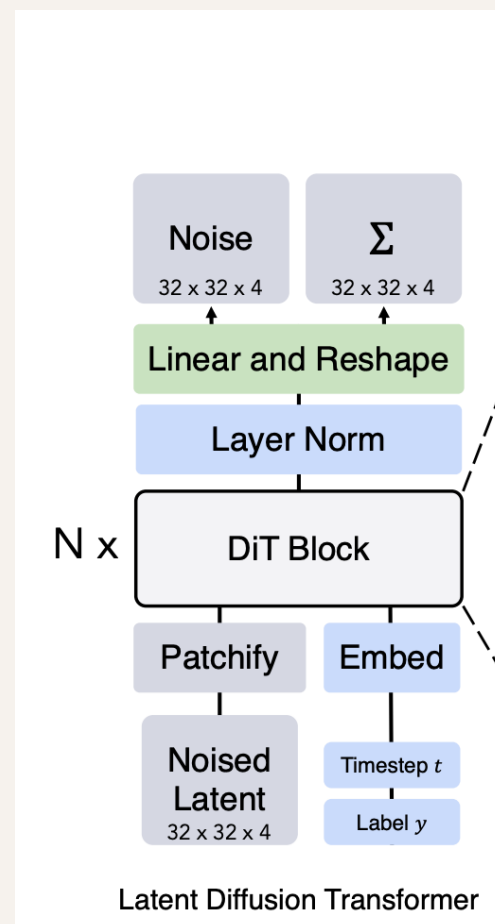
```
311 for block in self.blocks:
312     nn.init.constant_(block.adaLN_modulation[-1].weight, 0)
313     nn.init.constant_(block.adaLN_modulation[-1].bias, 0)
314
315     nn.init.constant_(self.final_layer.adaLN_modulation[-1].weight, 0)
316     nn.init.constant_(self.final_layer.adaLN_modulation[-1].bias, 0)
```

$$\frac{\partial L}{\partial W} = \frac{\partial L}{\partial \alpha} \cdot \frac{\partial \alpha}{\partial W} = \frac{\partial L}{\partial \alpha} \cdot c^T$$



2 DiT – Method

d. Code of Model pipeline and output



```
330 def forward(  
331     self, x: torch.Tensor, t: torch.Tensor, y: torch.Tensor  
332 ) -> torch.Tensor:  
333     """(B,C,H,W) noisy latent -> noise prediction."""  
334     x = self.x_embedder(x) + self.pos_embed  
335     t = self.t_embedder(t)  
336     y = self.y_embedder(y, self.training)  
337     c = t + y  
338     for block in self.blocks:  
339         x = block(x, c)  
340     x = self.final_layer(x, c)  
341     return self.unpatchify(x)
```


2 DiT – Method

e. Code of Loss Function

```
425 def _vb_terms_bpd(self, model, x_start, x_t, t, clip_denoised=True, model_kwargs=None):
426     """Compute VLB terms (bits per dimension)."""
427     true_mean, _, true_log_variance_clipped = self.q_posterior_mean_variance(
428         x_start=x_start, x_t=x_t, t=t
429     )
430     out = self.p_mean_variance(
431         model, x_t, t, clip_denoised=clip_denoised, model_kwargs=model_kwargs
432     )
433     kl = normal_kl(true_mean, true_log_variance_clipped, out["mean"], out["log_variance"])
434     kl = mean_flat(kl) / np.log(2.0)
435     # Use discrete log-likelihood at t=0
436     decoder_nll = -discretized_gaussian_log_likelihood(
437         x_start, means=out["mean"], log_scales=0.5 * out["log_variance"]
438     )
439     assert decoder_nll.shape == x_start.shape
440     decoder_nll = mean_flat(decoder_nll) / np.log(2.0)
441     output = th.where((t == 0), decoder_nll, kl)
442     return {"output": output, "pred_xstart": out["pred_xstart"]}
```

```
109 class GaussianDiffusion:
444     def training_losses(self, model, x_start, t, model_kwargs=None, noise=None,
470         # Separate learned variance + VLB auxiliary loss
471         if self.model_var_type in [ModelVarType.LEARNED, ModelVarType.LEARNED_RANGE]:
472             B, C = x_t.shape[:2]
473             assert model_output.shape == (B, C * 2, *x_t.shape[2:])
474             model_output, model_var_values = th.split(model_output, C, dim=1)
475             # Detach so variance prediction doesn't affect mean prediction
476             frozen_out = th.cat([model_output.detach(), model_var_values], dim=1)
477             terms["vb"] = self._vb_terms_bpd(
478                 model=lambda *args, r=frozen_out: r,
479                 x_start=x_start, x_t=x_t, t=t, clip_denoised=False,
480             )["output"]
481             if self.loss_type == LossType.RESCALED_MSE:
482                 terms["vb"] *= self.num_timesteps / 1000.0
```

$$L_{\epsilon-MSE} = \mathbb{E}_{t, x_0, \epsilon} [\|\epsilon - \epsilon_{\theta}(x_t, t)\|^2]$$

$$L_{vb} = \mathbb{E}_{t, x_0, \epsilon} [D_{KL}(q(x_{t-1}|x_t, x_0) \parallel p_{\theta}(x_{t-1}|x_t))]$$

$$L_{total} = L_{\epsilon-MSE} + \lambda L_{vb}$$

2 DiT – Method

f. Code of CFG Sampling

$$FID = |\mu_r - \mu_g|^2 + \text{Tr}(\Sigma_r + \Sigma_g - 2(\Sigma_r \Sigma_g)^{\frac{1}{2}})$$

To evaluate Model, we have to use FID Metric,
so we need to use CFG Sampling

```
343 def forward_with_cfg(  
344     self, x: torch.Tensor, t: torch.Tensor, y: torch.Tensor, cfg_scale: float  
345 ) -> torch.Tensor:  
346     """CFG sampling: linear combination of cond/uncond outputs."""  
347     half = x[: len(x) // 2]  
348     combined = torch.cat([half, half], dim=0)  
349     model_out = self.forward(combined, t, y)  
350     eps, rest = model_out[:, :self.in_channels], model_out[:, self.in_channels:]  
351     cond_eps, uncond_eps = torch.split(eps, len(eps) // 2, dim=0)  
352     guided_eps = uncond_eps + cfg_scale * (cond_eps - uncond_eps)  
353     eps = torch.cat([guided_eps, guided_eps], dim=0)  
354     return torch.cat([eps, rest], dim=1)
```

$$\hat{\epsilon}_{\theta}(x_t, c) = \epsilon_{\theta}(x_t, \emptyset) + w(\epsilon_{\theta}(x_t, c) - \epsilon_{\theta}(x_t, \emptyset))$$

2 DiT – Method

f. Code of CFG Sampling

```
144 class LabelEmbedder(nn.Module):
168     def forward(
174         """Replace labels with null token at dropout_prob during training."""
175         if (train and self.dropout_prob > 0) or (force_drop_ids is not None):
176             labels = self._drop_labels(labels, force_drop_ids)
177         return self.embedding_table(labels)
```

To enable CFG, we drop the class token to a null token 10% of the time during training to also learn unconditional generation.

2 DiT – Method

g. Code of sampling process

```
275 def p_sample(self, model, x, t, clip_denoised=True, denoised_fn=None,
276             cond_fn=None, model_kwargs=None):
277     """Single reverse step (DDPM)."""
278     out = self.p_mean_variance(
279         model, x, t, clip_denoised=clip_denoised,
280         denoised_fn=denoised_fn, model_kwargs=model_kwargs,
281     )
282     noise = th.randn_like(x)
283     # No noise added at t=0
284     nonzero_mask = (t != 0).float().view(-1, *[1] * (len(x.shape) - 1))
285     if cond_fn is not None:
286         out["mean"] = self.condition_mean(cond_fn, out, x, t, model_kwargs=model_kwargs)
287     sample = out["mean"] + nonzero_mask * th.exp(0.5 * out["log_variance"]) * noise
288     return {"sample": sample, "pred_xstart": out["pred_xstart"]}
289
290 def p_sample_loop(self, model, shape, noise=None, clip_denoised=True,
291                  denoised_fn=None, cond_fn=None, model_kwargs=None,
292                  device=None, progress=False):
293     """Full T->0 reverse sampling loop."""
294     final = None
295     for sample in self.p_sample_loop_progressive(
296         model, shape, noise=noise, clip_denoised=clip_denoised,
297         denoised_fn=denoised_fn, cond_fn=cond_fn,
298         model_kwargs=model_kwargs, device=device, progress=progress,
299     ):
300         final = sample
301     return final["sample"]
```

$$x_{t-1} = \mu_{\theta}(x_t, t) + \sigma_{\theta} \cdot z$$

2 DiT – Method

g. Code of sampling process

```
44 class SpacedDiffusion(GaussianDiffusion):
45     """Respaced diffusion – uses only selected timesteps."""
46
47     def __init__(self, use_timesteps, **kwargs):
48         self.use_timesteps = set(use_timesteps)
49         self.timestep_map = []
50         self.original_num_steps = len(kwargs["betas"])
51
52         # Recompute betas from selected alpha_bar values in original schedule
53         base_diffusion = GaussianDiffusion(**kwargs)
54         last_alpha_cumprod = 1.0
55         new_betas = []
56         for i, alpha_cumprod in enumerate(base_diffusion.alphas_cumprod):
57             if i in self.use_timesteps:
58                 new_betas.append(1 - alpha_cumprod / last_alpha_cumprod)
59                 last_alpha_cumprod = alpha_cumprod
60                 self.timestep_map.append(i)
61         kwargs["betas"] = np.array(new_betas)
62         super().__init__(**kwargs)
```

3 DiT – Experiment

a. Scaling law over model parameters

Model	Layers N	Hidden size d	Heads	Gflops ($I=32, p=4$)
DiT-S	12	384	6	1.4
DiT-B	12	768	12	5.6
DiT-L	24	1024	16	19.7
DiT-XL	28	1152	16	29.1

Figure 6. Model-size settings (DiT-S/B/L/XL) used for the parameter scaling study. Adapted from Peebles, W., & Xie, S. (2023). Scalable diffusion models with transformers. In Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV) (pp. 4195-4205).

3 DiT – Experiment

b. Scaling law over patch size

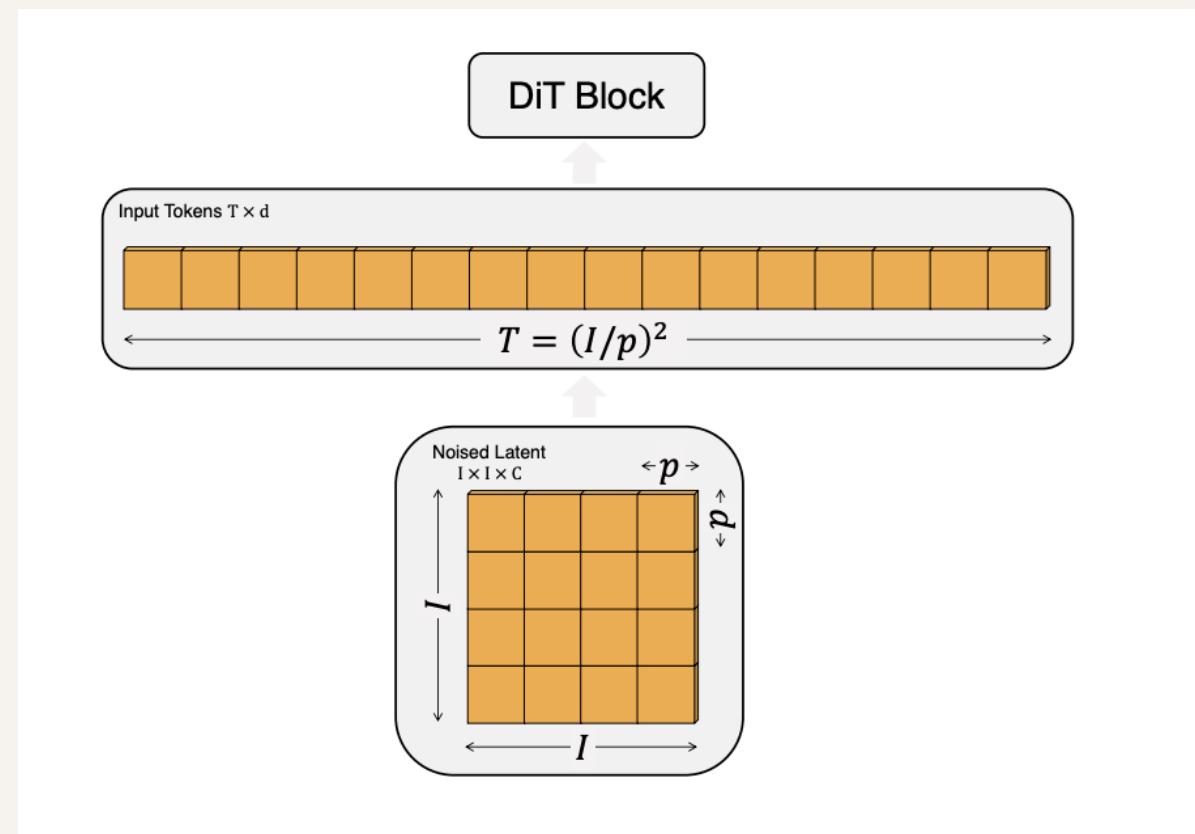


Figure 6. Patchify input specification for DiT. A smaller patch size p gives a longer token sequence and thus more GFLOPs. Adapted from Peebles, W., & Xie, S. (2023). Scalable diffusion models with transformers. In *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)* (pp. 4195-4205).

4 DiT – Conclusion

a. Scaling law over model parameters

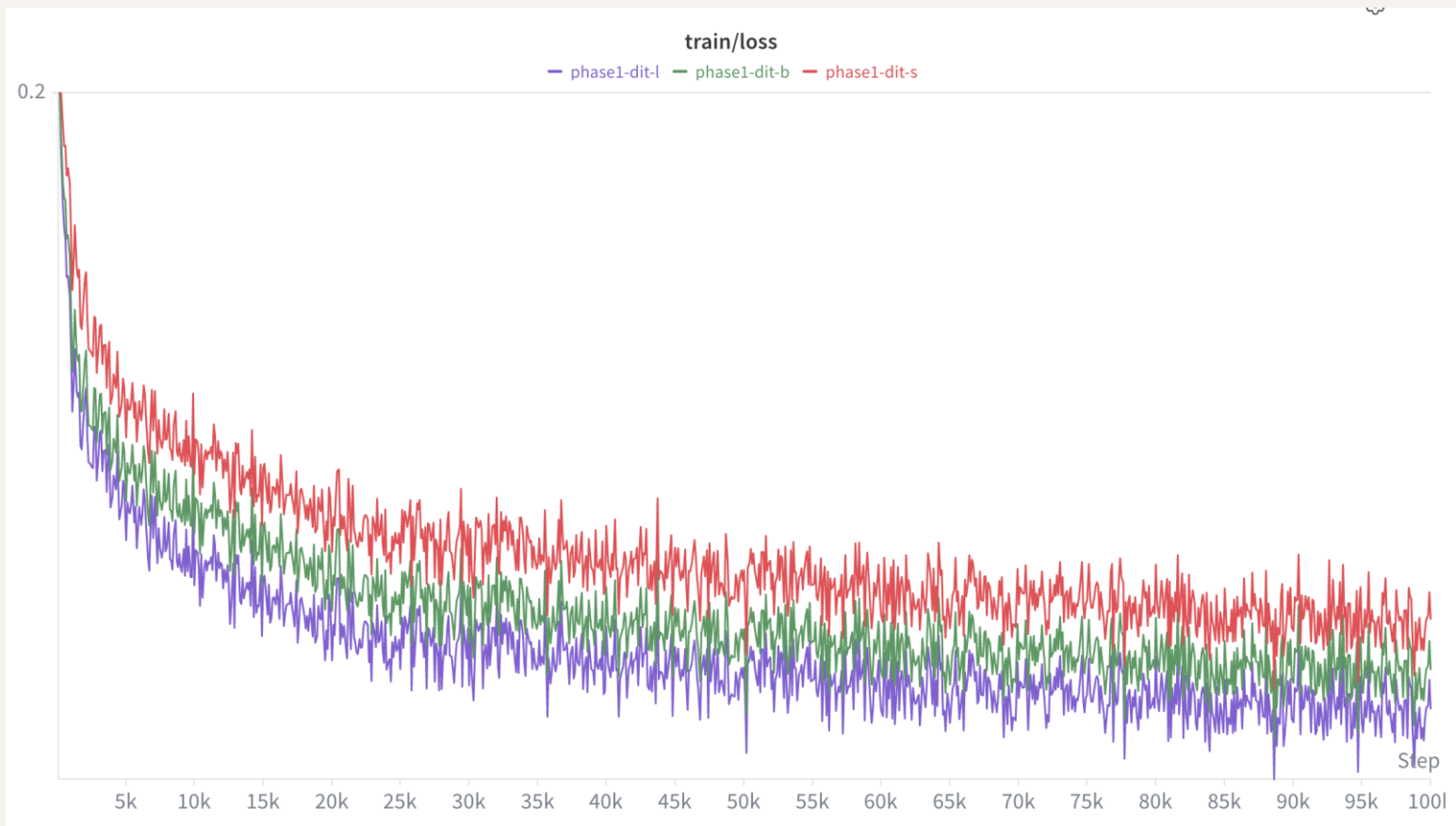


Figure 7. Training loss over steps for DiT-S/B/L. Larger models converge faster. Source: own work.

# core	Model	# parameter	step/sec	final train loss(MSE) ↓	FID ↓
2	DiT-s	33M	6.7	0.156	61.5
2	DiT-b	130M	2.8	0.153	35.9
2	DiT-l	458M	0.9	0.15	14.71

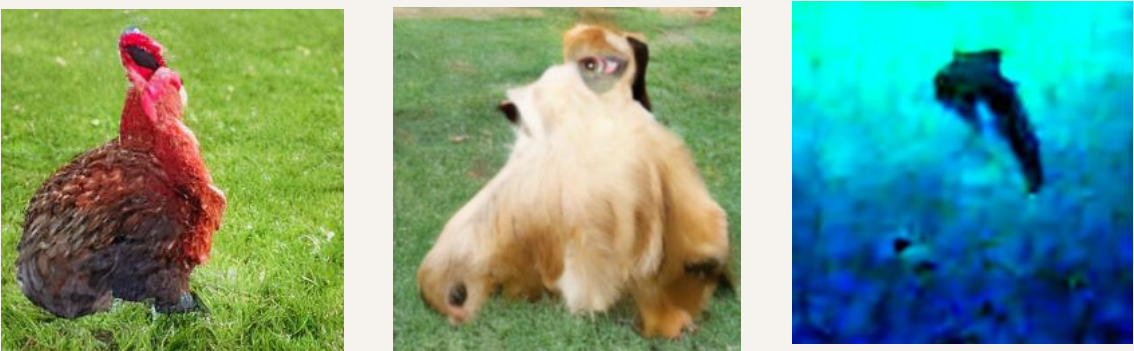


Figure 8. Class-conditional samples from L, B, and S size DiT (left to right). Sample quality rises with model size. Source: own work.

4 DiT – Conclusion

b. Scaling law over patch size



# core	Model	Patch size	step/sec	final train loss(MSE) ↓	FID ↓
2	DiT-s/2	2	6.7	0.156	61.5
2	DiT-s/4	4	17.8	0.164	109.19
2	DiT-s/8	8	18.4	0.175	115.05

Figure 9. Training loss for patch sizes 2, 4, 8. Smaller patch (more GFLOPs) converges faster.
Source: own work.

4 DiT – Conclusion

GFlops	GFlops(log)	FID ↓
19.7	1.294	14.71
5.6	0.748	35.9
1.4	0.146	61.5
0.4	-0.398	109.19
0.1	-1	115.05

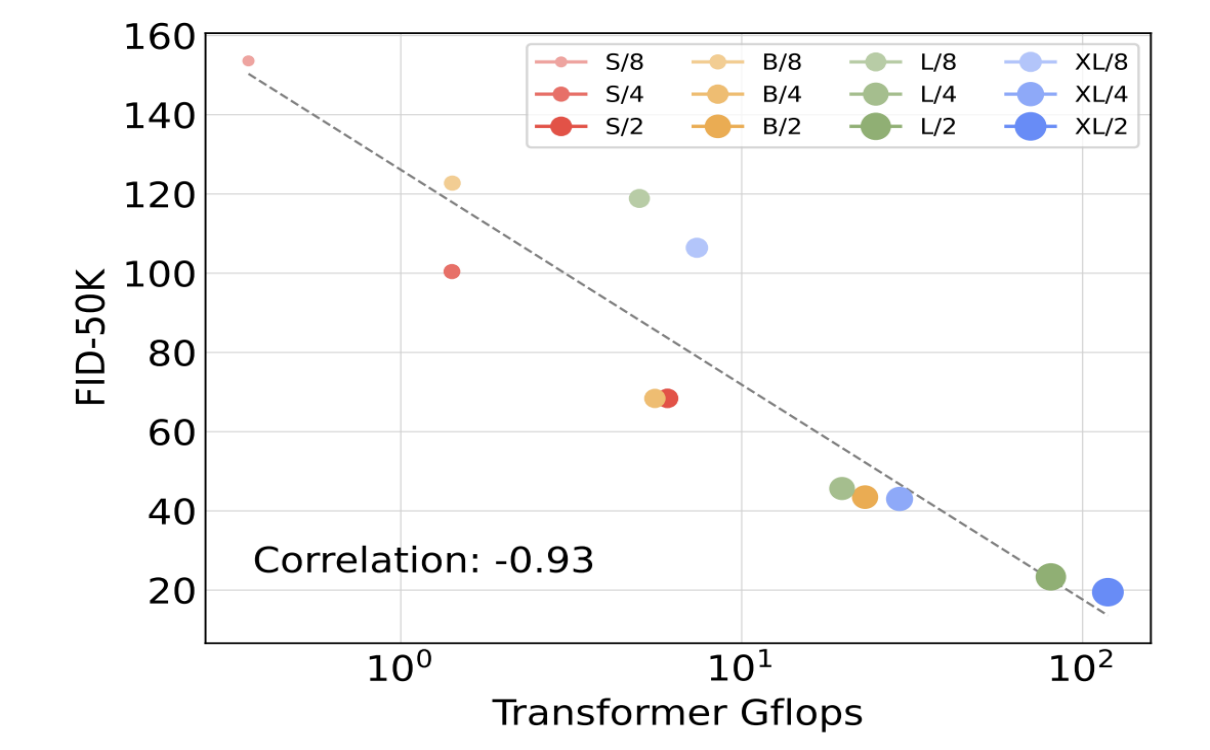
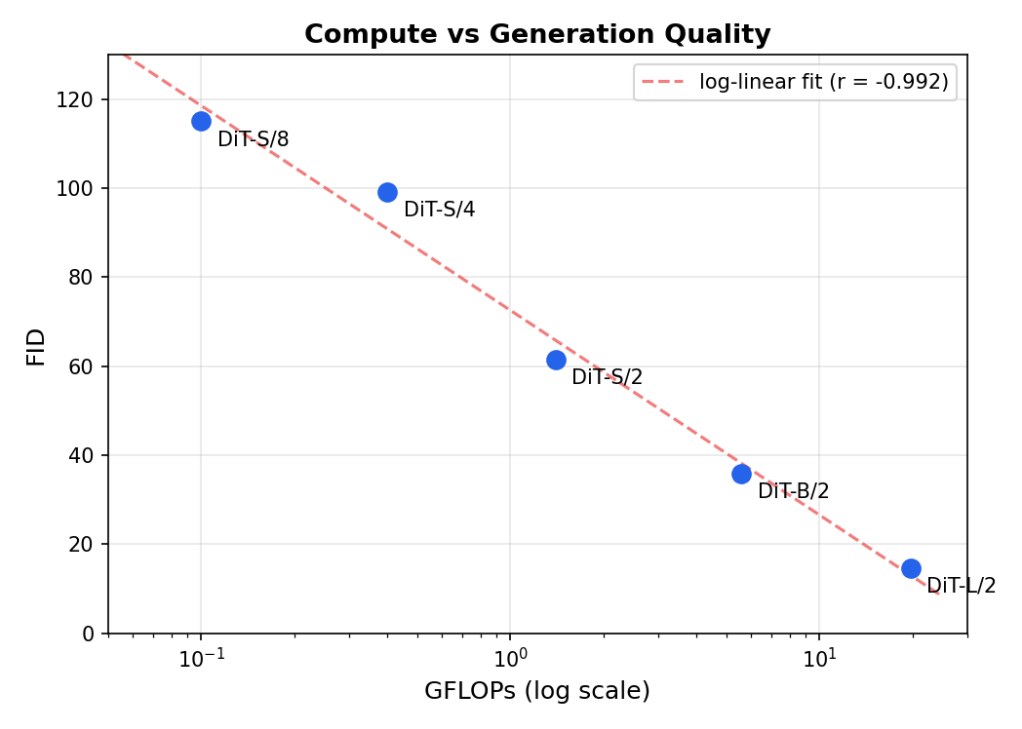


Figure 10. Compute versus generation quality. Left: our DiT runs, log-GFLOPs versus FID, with a log-linear fit ($r = -0.992$). Right: the original DiT trend (corr -0.93) for comparison. Left panel: own work; right panel adapted from Peebles, W., & Xie, S. (2023). Scalable diffusion models with transformers. In *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)* (pp. 4195-4205).

HiT–Hyperbolic DiT

5 HiT – Intuition & Hypothesis

Intuition 1:
Diffusion Model's latent space encodes everything from data to full noise as one continuous structure.

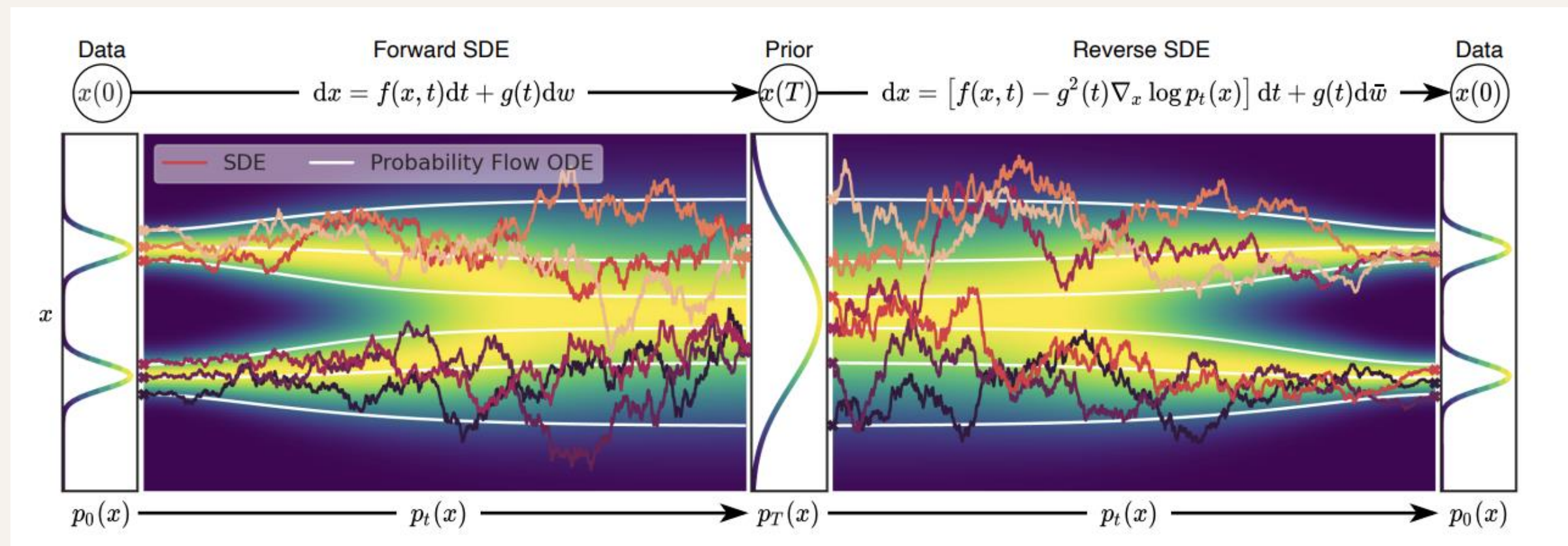


Figure 11. Score-based diffusion as a continuous-time SDE that interpolates between data and a Gaussian. Motivates viewing all noise levels as one continuous structure. Adapted from Song, Y., Sohl-Dickstein, J., Kingma, D. P., Kumar, A., Ermon, S., & Poole, B. (2021). Score-based generative modeling through stochastic differential equations. In International Conference on Learning Representations (ICLR).

5 HiT – Intuition & Hypothesis

Intuition 2:
Any data, when fully noised, converges to the same Gaussian.

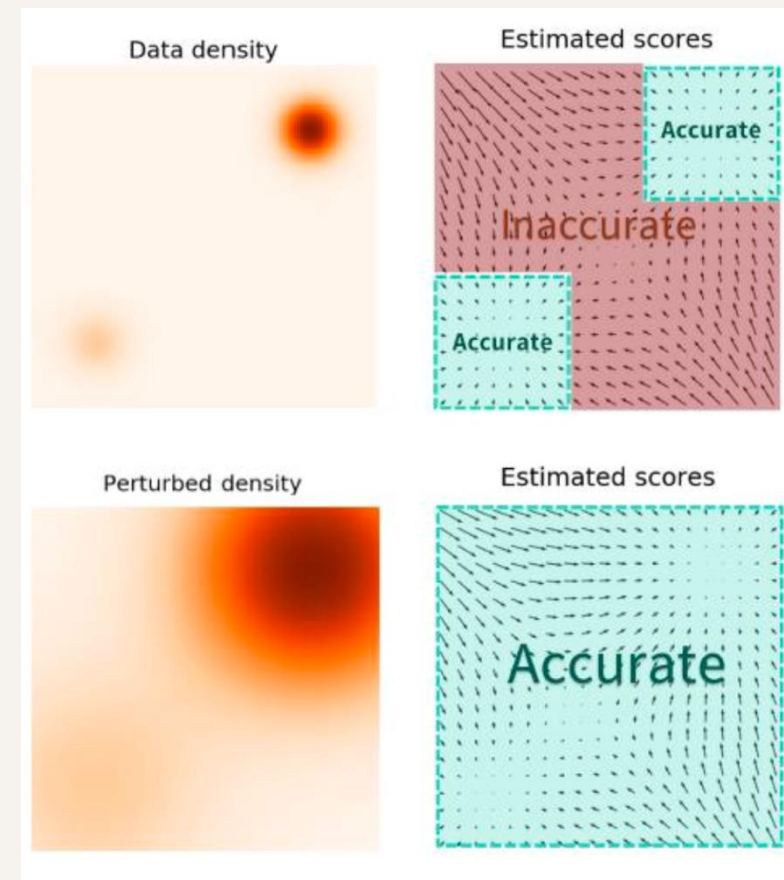


Figure 12. A single model learns scores across a continuous family of noise-smoothed distributions. Adapted from Song, Y., & Ermon, S. (2019). Generative modeling by estimating gradients of the data distribution. In Advances in Neural Information Processing Systems (NeurIPS) (Vol. 32, pp. 11918-11930).

5 HiT – Intuition & Hypothesis

Intuition 3:
Better-trained model's latent state converge to stable fixed points and follow consistent trajectories.

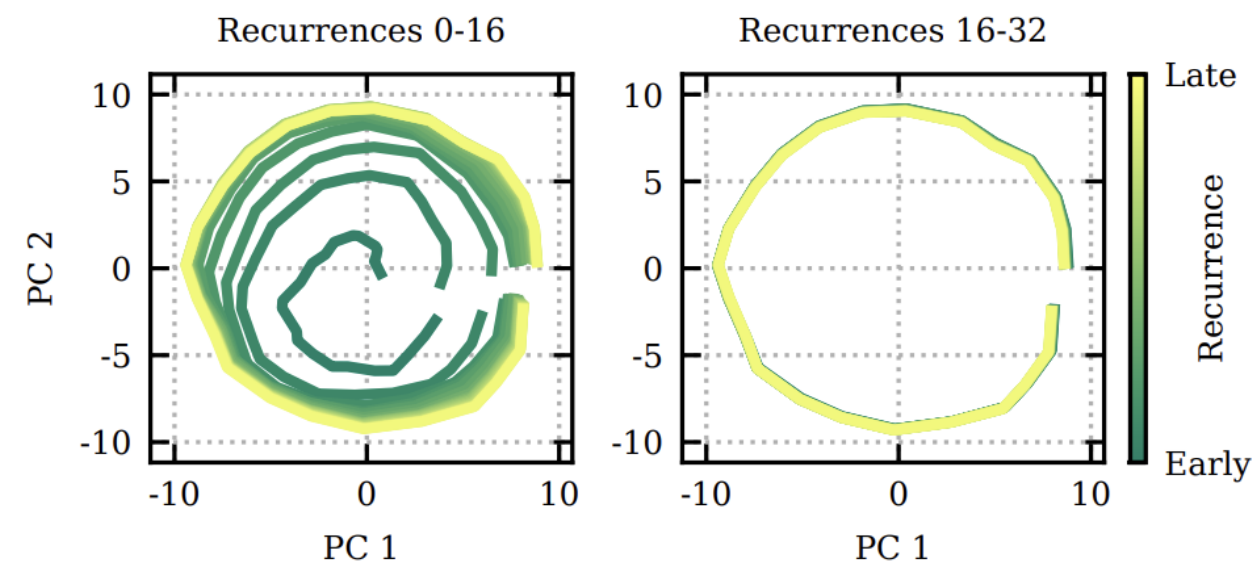


Figure 1. Latent states after each block in a recurrent model frequently tend towards *separate* fixed points, meaning that the application of a recurrent block tends towards a consistent trajectory in latent space.

Figure 13. In a weight-tied recurrent model, latent states tend toward consistent trajectories and fixed points. Adapted from Blayney, H., Arroyo, Á., Obando-Ceron, J., Castro, P. S., Courville, A., Bronstein, M. M., & Dong, X. (2026). A mechanistic analysis of looped reasoning language models. arXiv. <https://arxiv.org/abs/2604.11791>

5 HiT – Intuition & Hypothesis

Our Hypothesis:
A well-trained diffusion's latent space forms a radial manifold with noise at the center and data spreading outward.

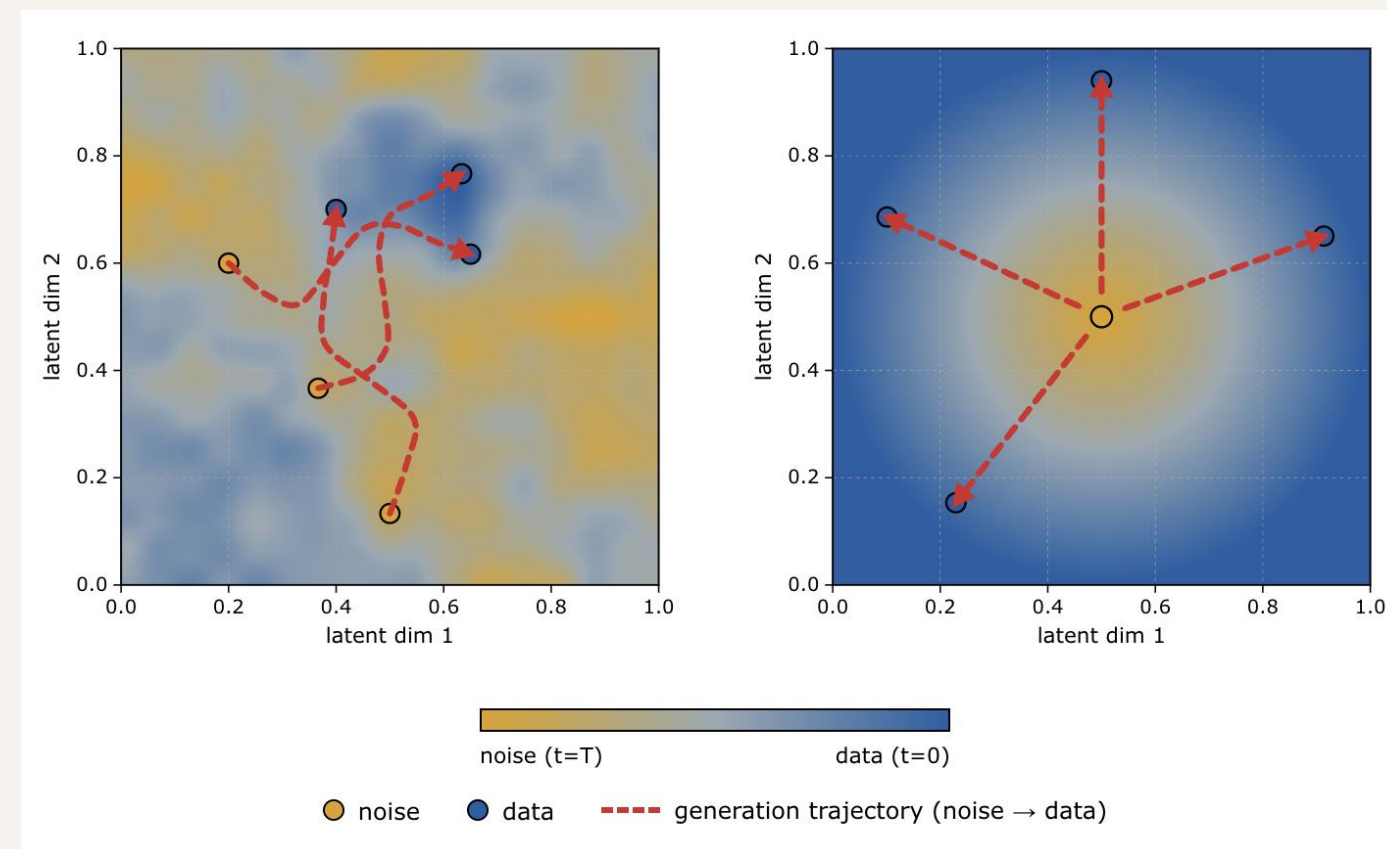


Figure 14. Intuition for the hypothesis. Both panels share one noise-to-data color map. Left: a disordered latent where trajectories wander. Right: a radial latent (noise at center, data at the boundary) where trajectories run straight. Source: own work.

6 HiT – Preliminary

Hyperbolic space: the non-Euclidean space that fits on hierarchical data

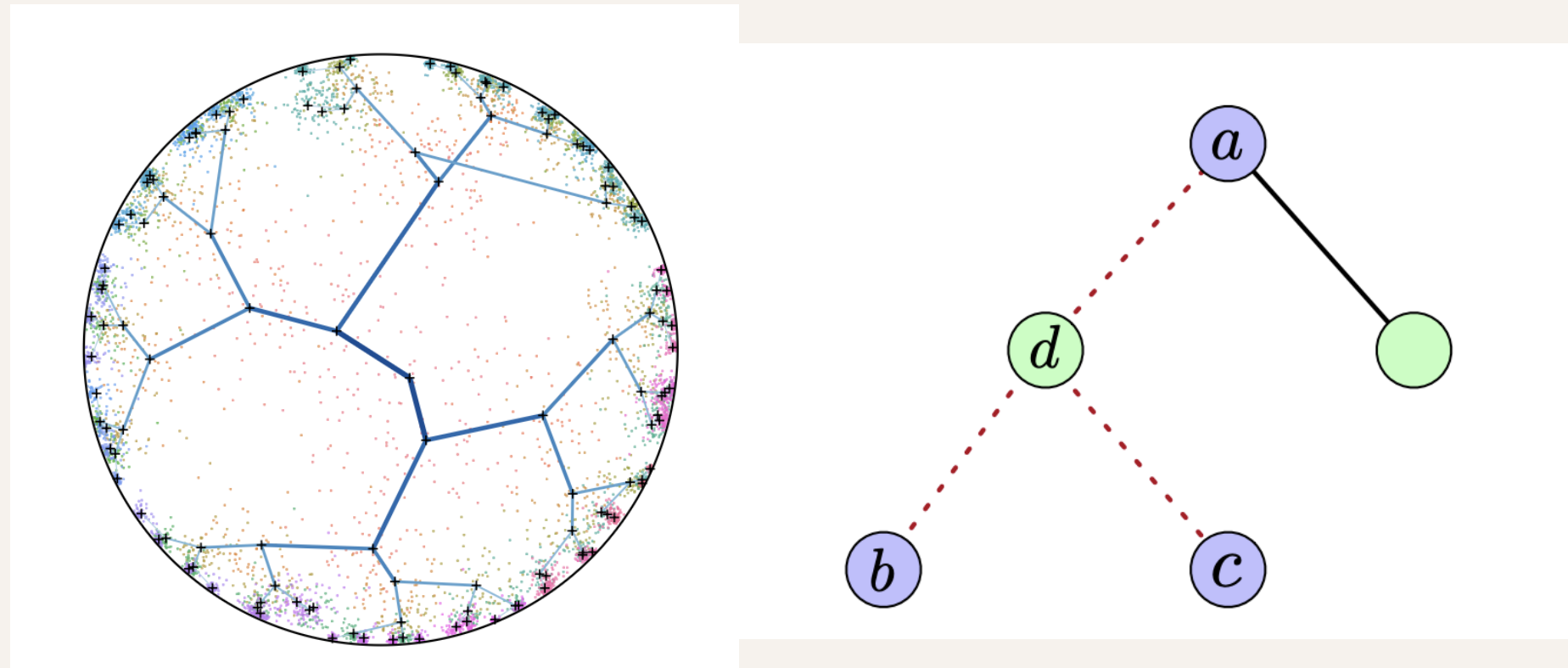


Figure 14. Intuition for the hypothesis. Both panels share one noise-to-data color map. Left: a disordered latent where trajectories wander. Right: a radial latent (noise at center, data at the boundary) where trajectories run straight. Source: H-VAE paper.

$$\exp_{\mathbf{x}}^c(\mathbf{v}) := \mathbf{x} \oplus_c \left(\tanh \left(\sqrt{c} \frac{\lambda_{\mathbf{x}}^c \|\mathbf{v}\|}{2} \right) \frac{\mathbf{v}}{\sqrt{c} \|\mathbf{v}\|} \right).$$

6 HiT – Preliminary

We can control model's dynamics to control last layer's latent space.

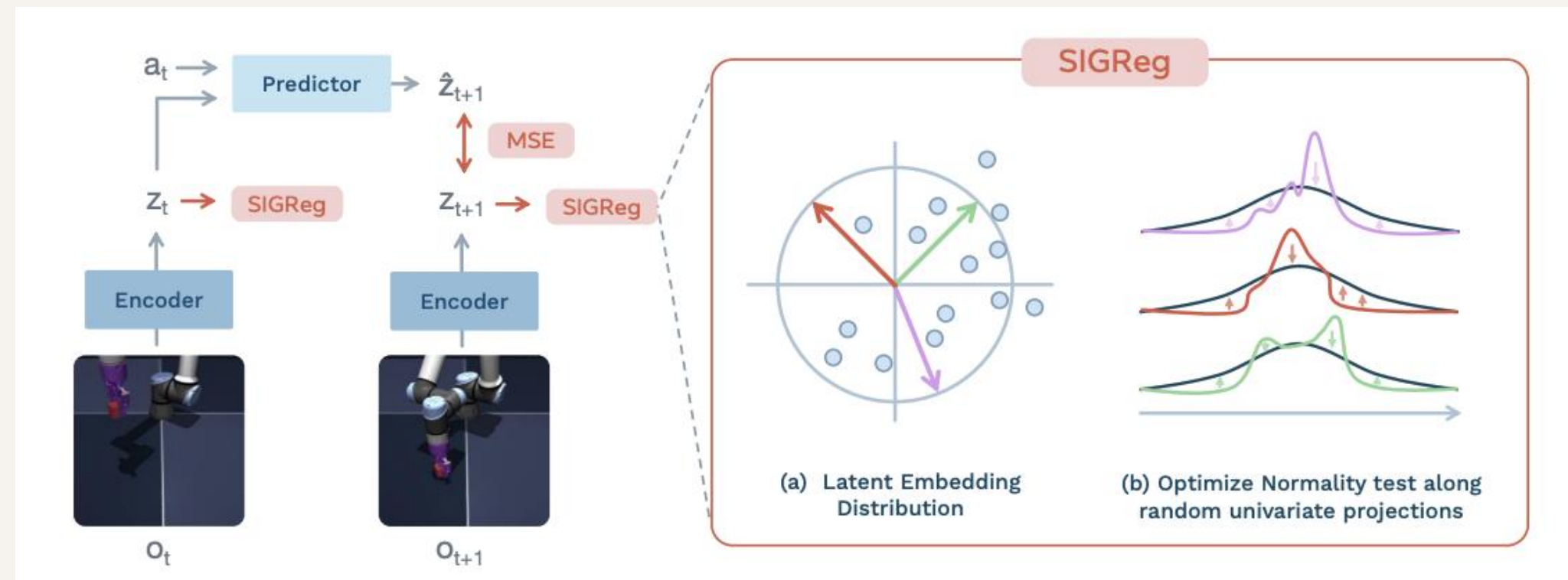


Figure 16. LeWorldModel: controlling the last-layer latent controls the whole model's dynamics, a precedent for our latent-level control. Adapted from LeCun, Y., Maes, L., Le Lidec, S., Scieur, D., & Balestriero, R. (2026). LeWorldModel. arXiv. <https://arxiv.org/abs/2603.19312>

7 HiT – Method

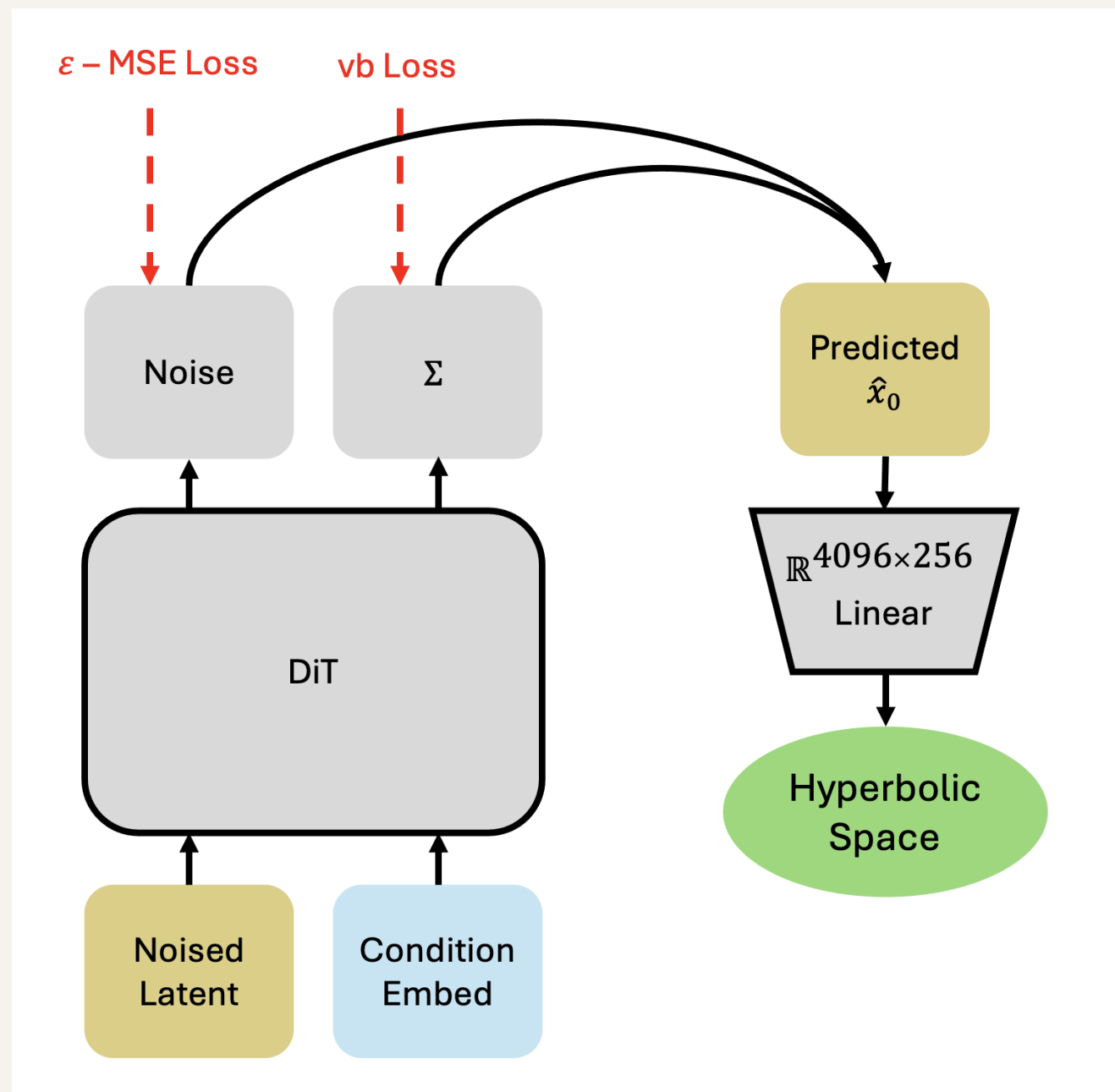
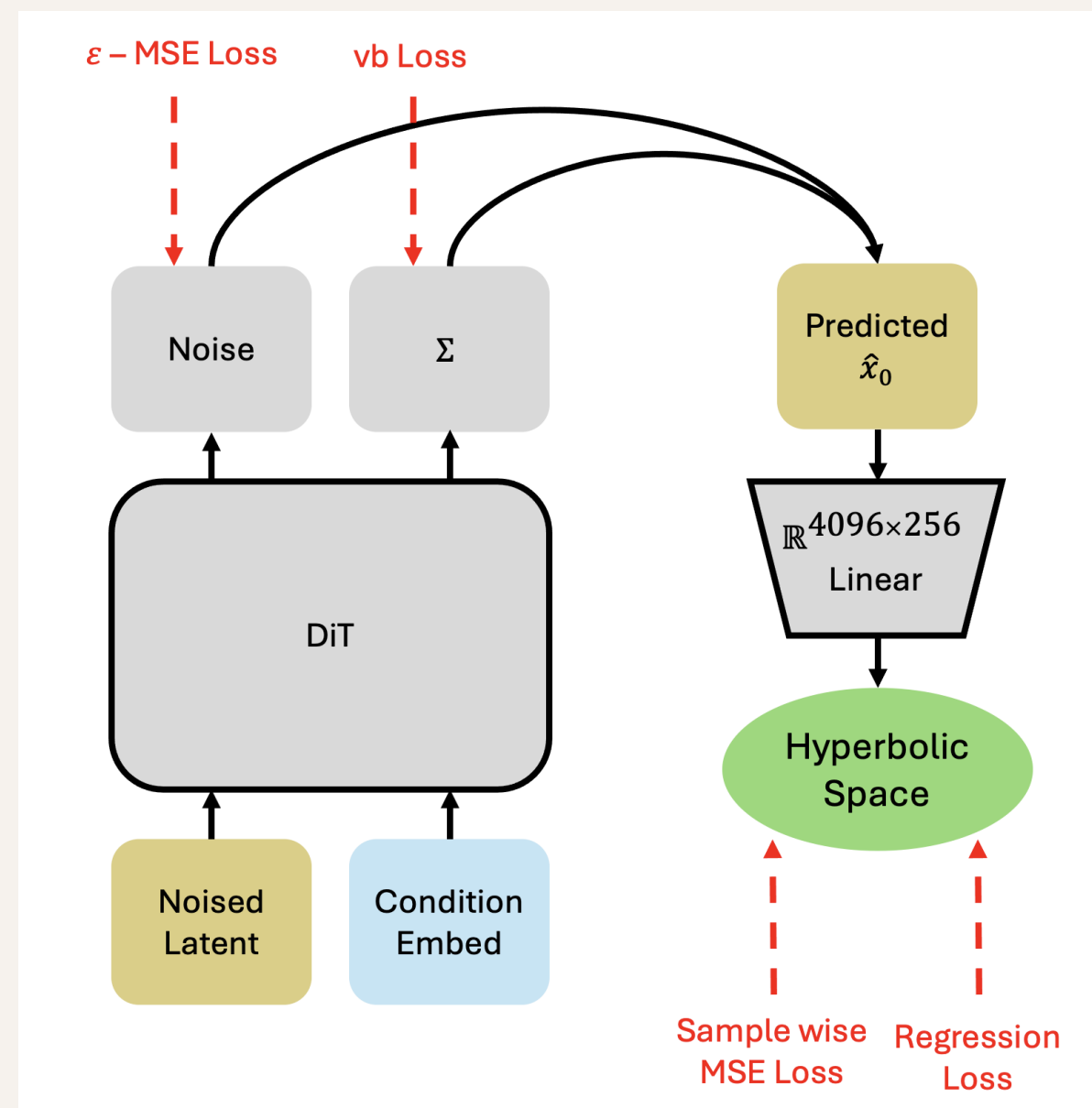


Figure 17. Our HiT pipeline. From the DiT output we reconstruct $\text{pred_}x_0$, project 4096 to 256 dimensions, and lift it onto hyperbolic space. Source: own work.

```
212 # ---- 프로젝션 헤드 (Stiefel 다양체) ----
213 #  $\epsilon$  예측(4096차원)을 저차원(256)으로 투영 후 쌍곡 공간 매핑
214 # 가중치: 직교행렬 제약 (QR 리트랙션으로 매 스텝 복원)
215 proj_head = None
216 use_proj = cfg.get("enabled", False)
217 proj_dim = cfg.get("dim", 256)
218 if use_proj:
219     flat_dim = raw_model.in_channels * latent_size * latent_size
220     proj_head = nn.Linear(flat_dim, proj_dim, bias=False).to(device)
221     nn.init.orthogonal_(proj_head.weight)
222     # 모든 랭크 동일 초기화 보장
223     if dist.is_initialized():
224         dist.broadcast(proj_head.weight.data, src=0)
225     logger.info(f"proj_head: {flat_dim} → {proj_dim} (Stiefel)")
226
```

7 HiT – Method



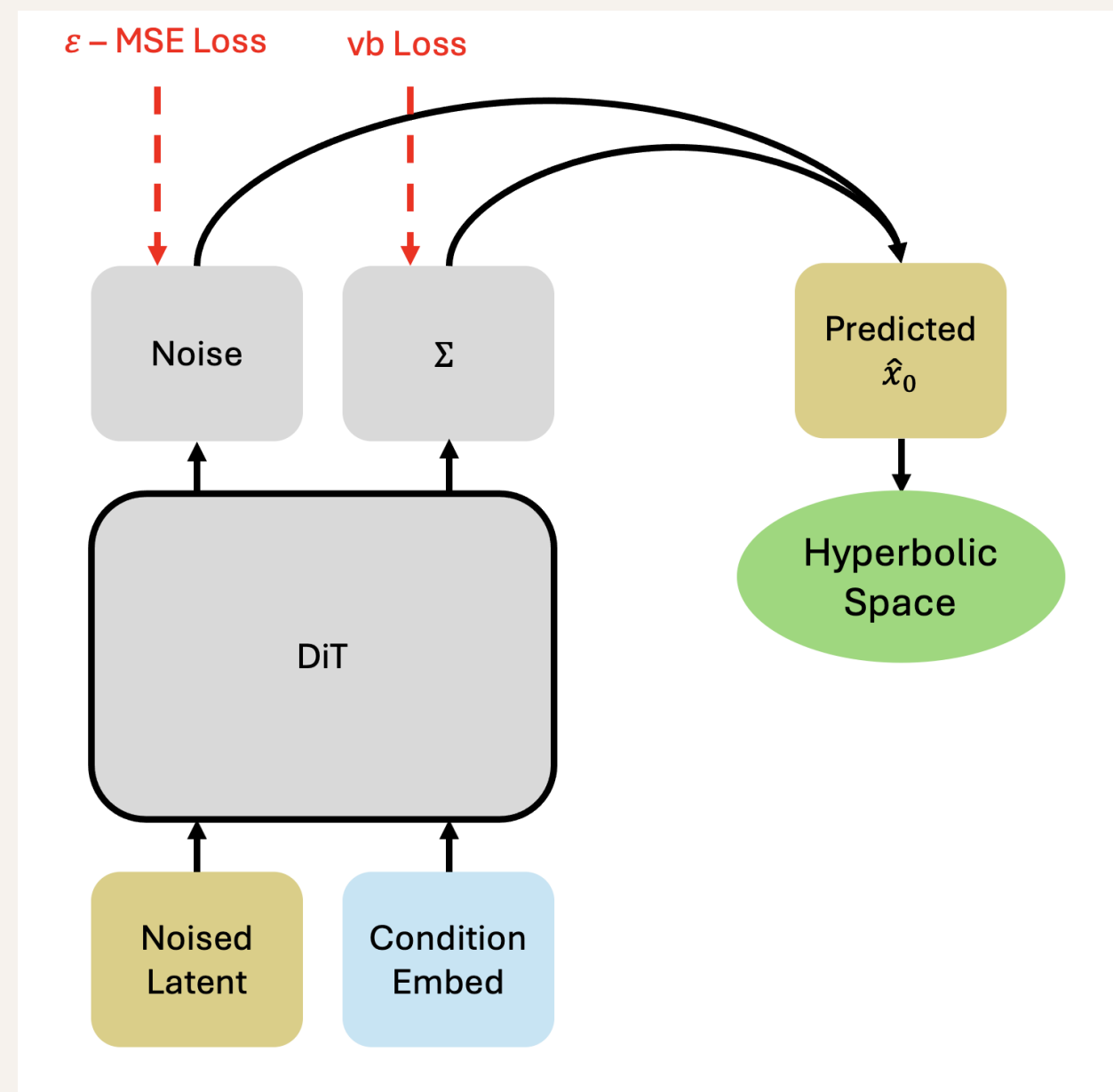
$$d(z, z^*) = \left(\frac{2}{\sqrt{c}} \right) \cdot \text{artanh}(\sqrt{c} \| \cdot \| \ominus z^* \oplus z)$$

$$L_{hyp} = E[d(z, z^*)^2]$$

$$L_{reg} = E \left[\left(d^0(z) - 2 \left(1 - \frac{t}{T} \right) \right)^2 \right]$$

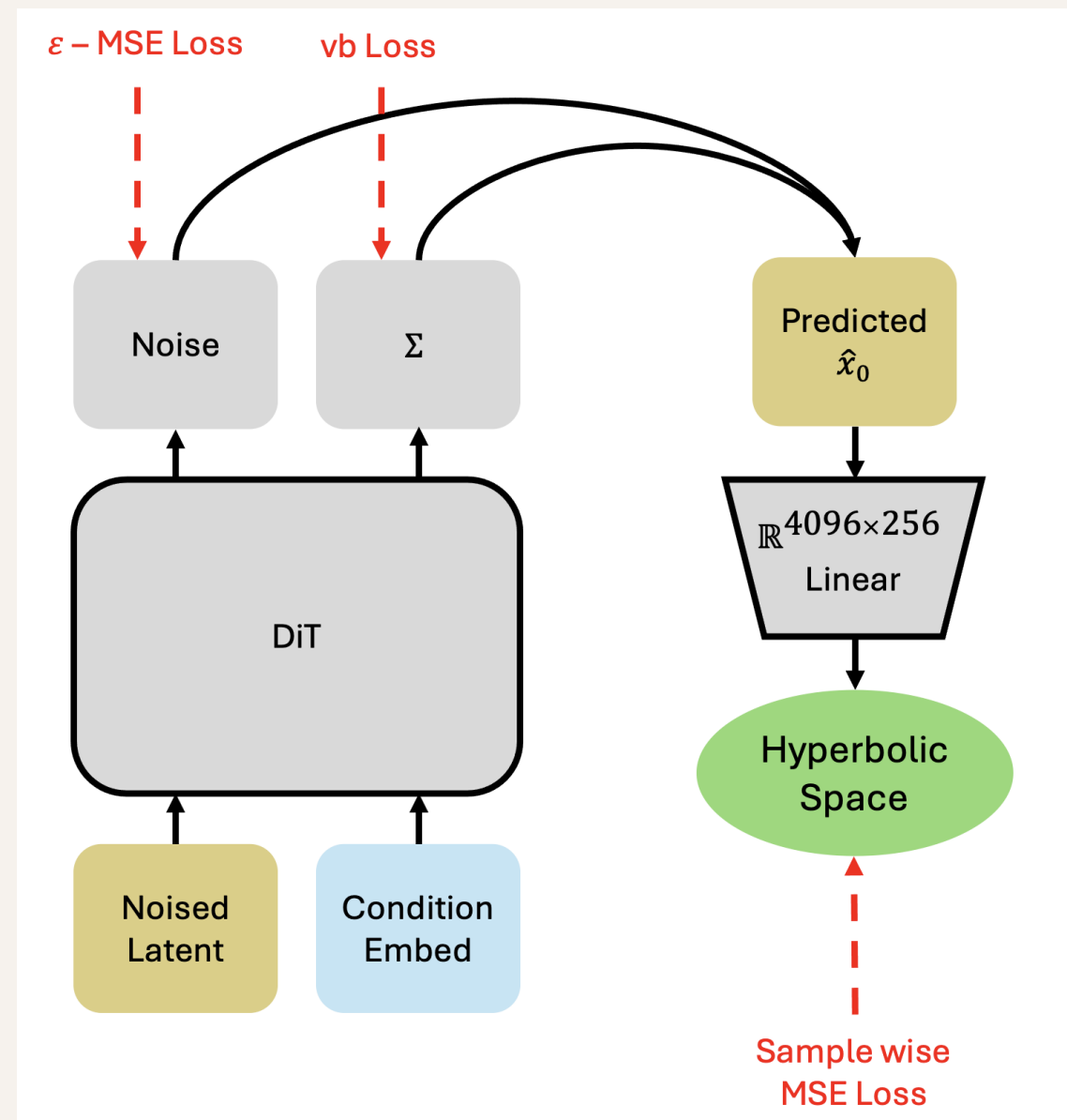
8 HiT – Experiment

a. Observe Vanilla DiT's Dynamics with Hyperbolic head



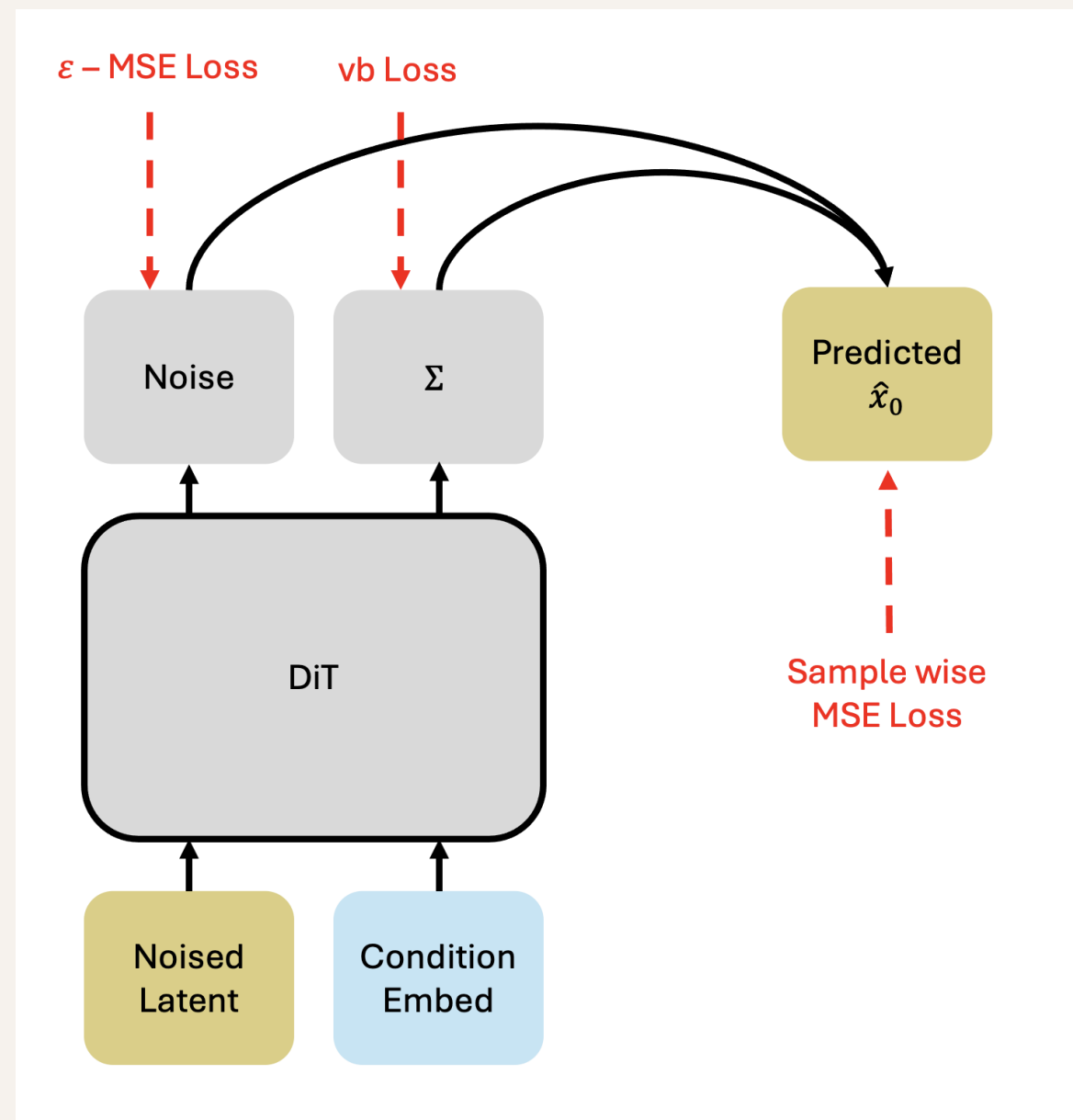
8 HiT – Experiment

b. Light intervention & Ablation



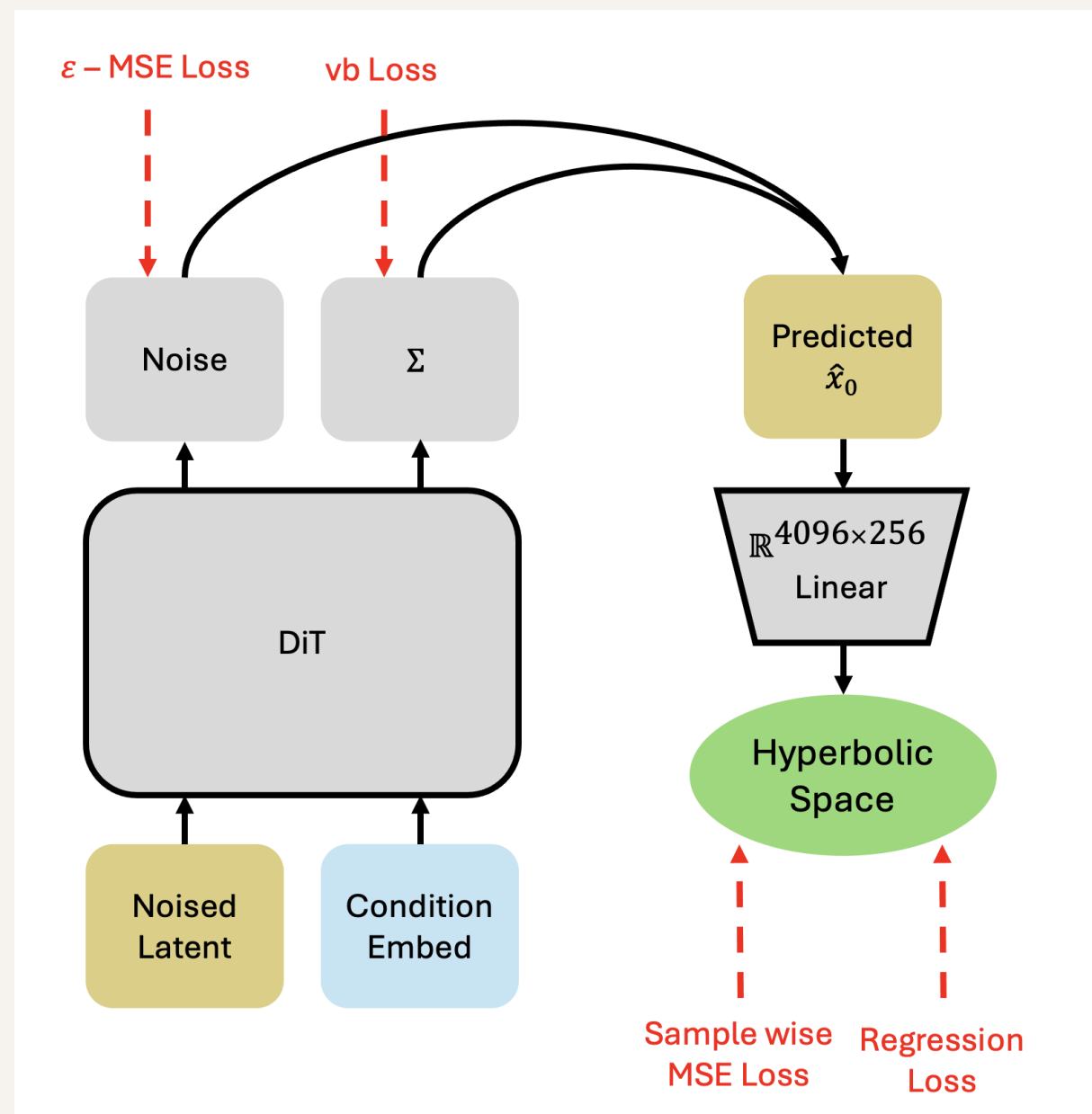
8 HiT – Experiment

b. Light intervention & Ablation



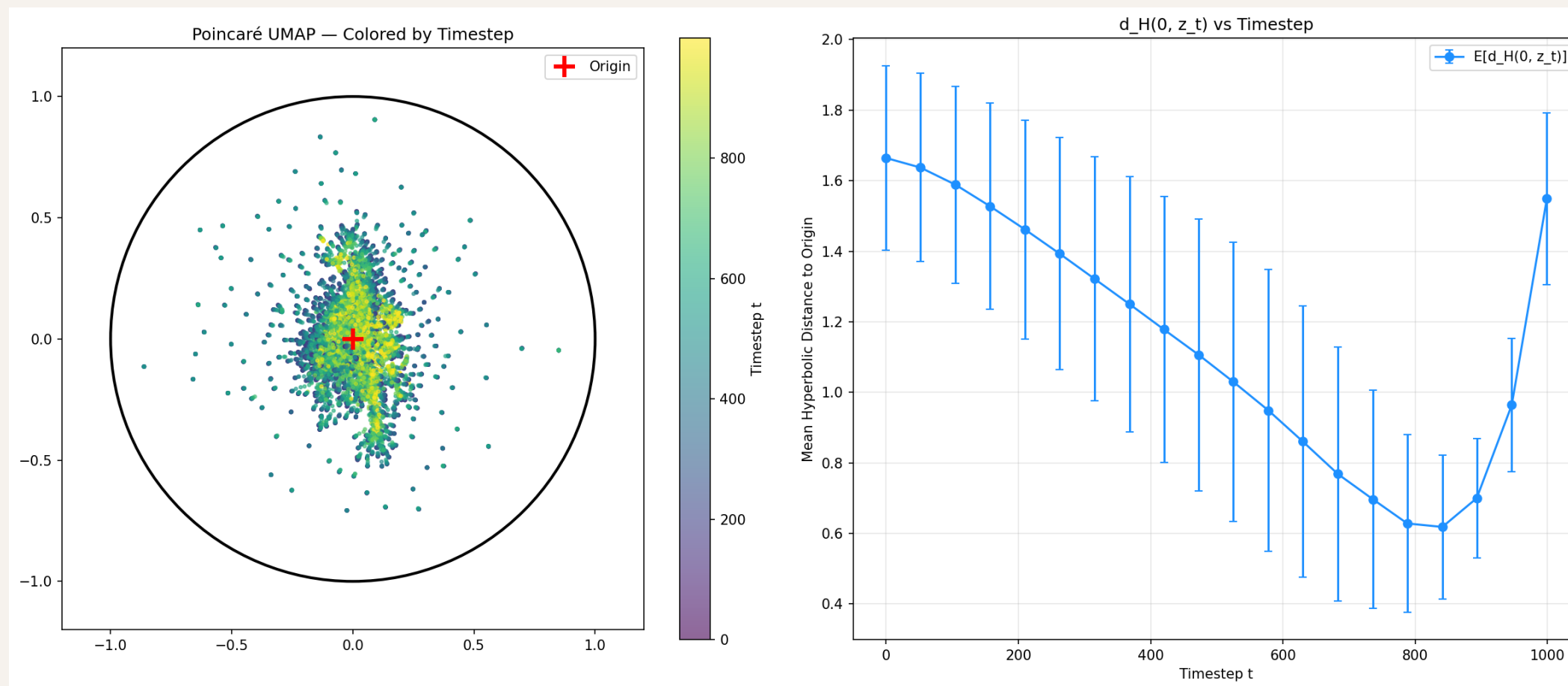
8 HiT – Experiment

c. Strong Intervention



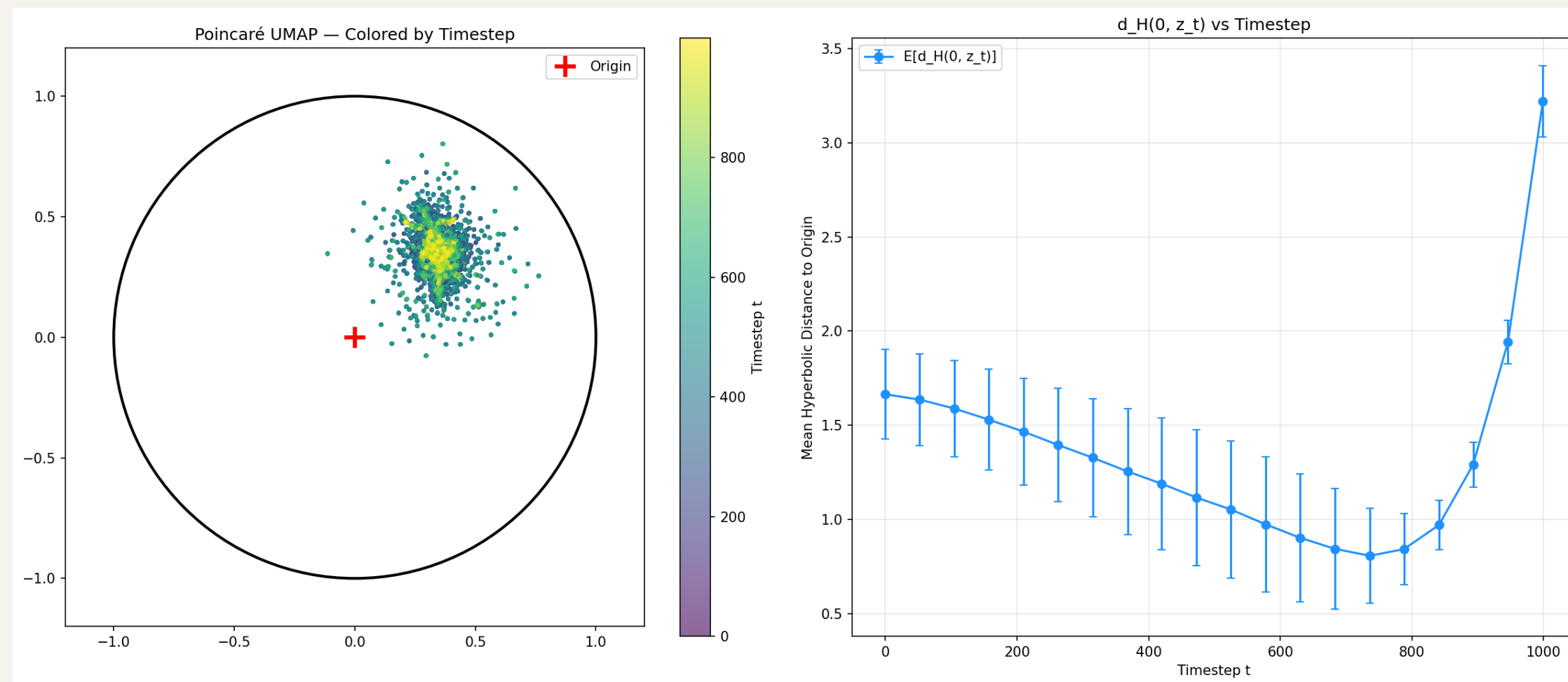
9 HiT – Conclusion

a. Observe Vanilla DiT's Dynamics with Hyperbolic head



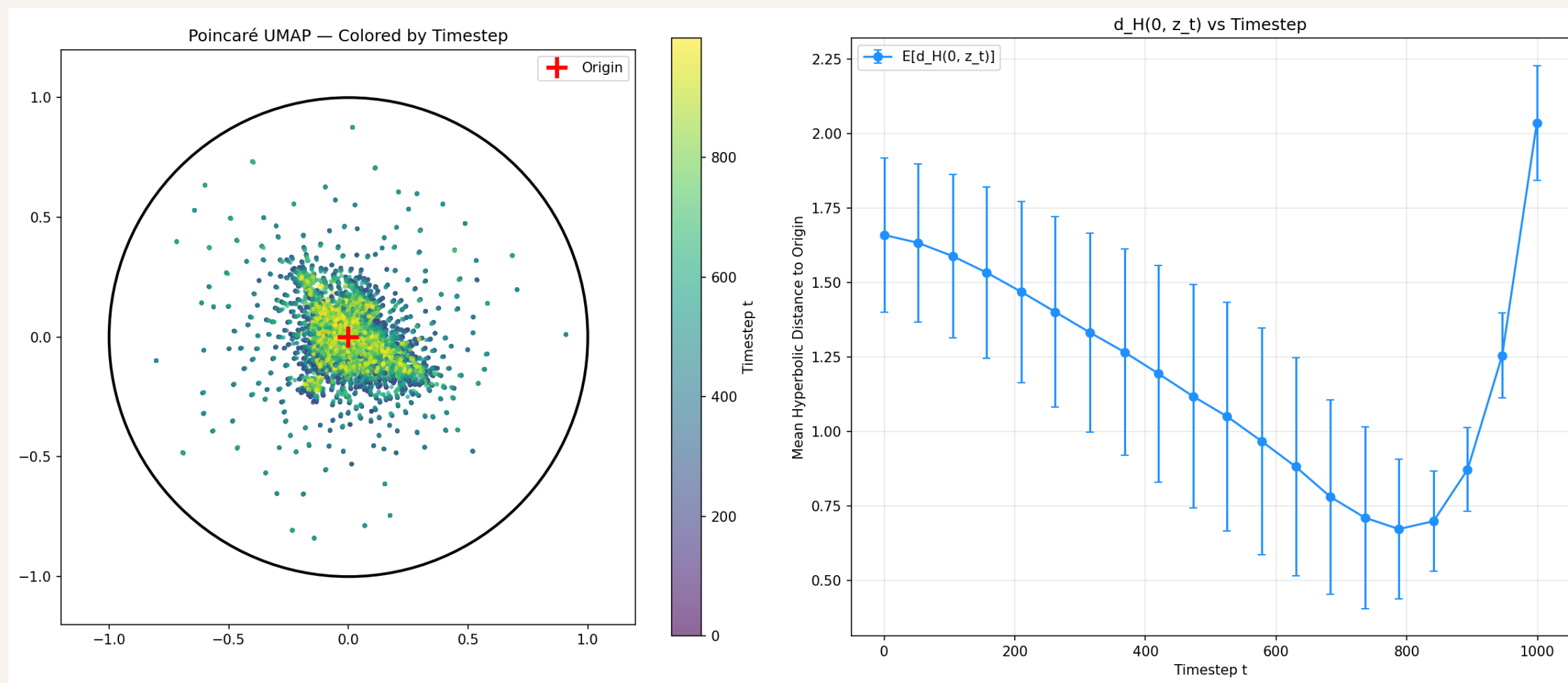
9 HiT – Conclusion

a. Observe Vanilla DiT's Dynamics with Hyperbolic head



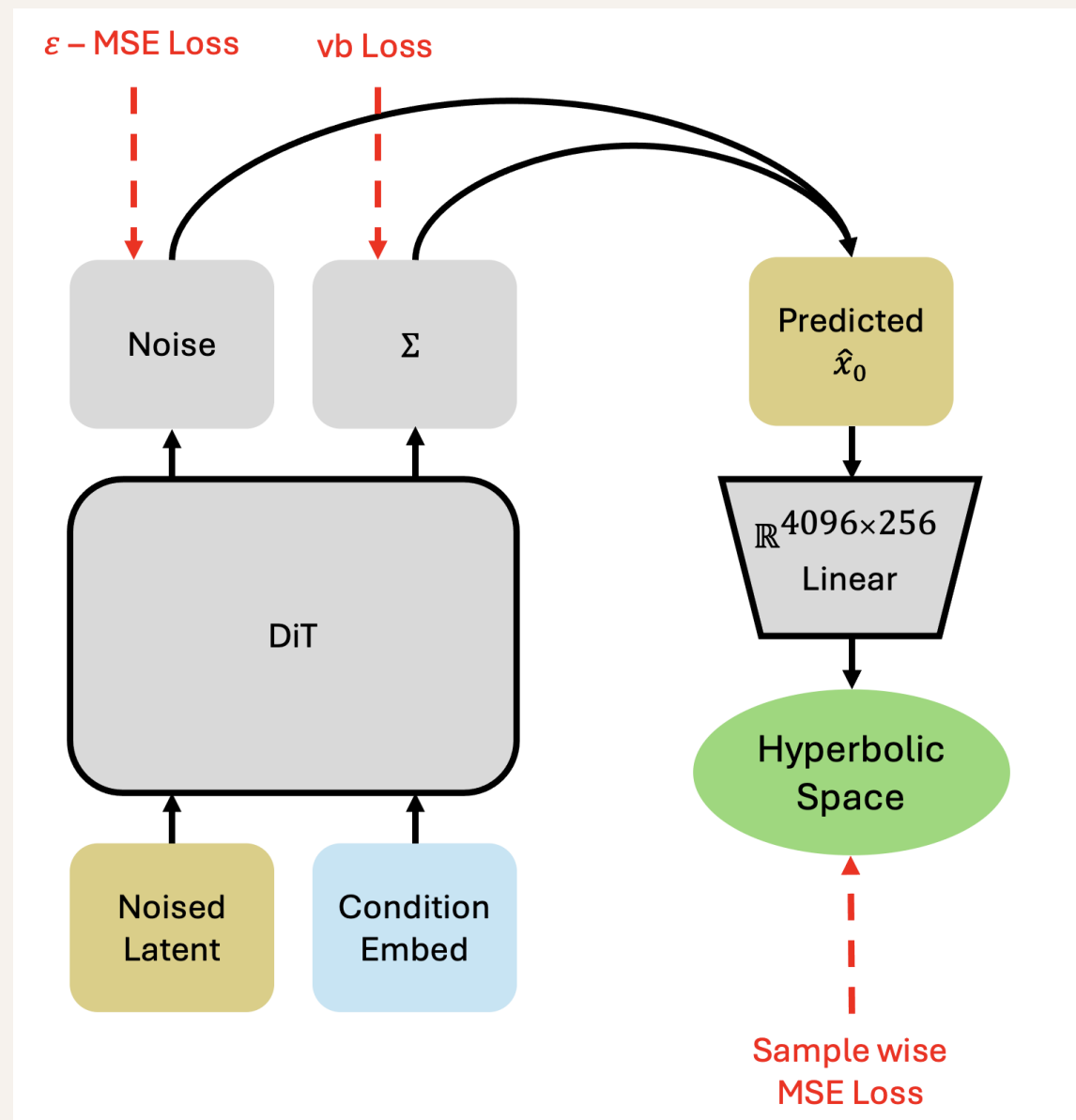
9 HiT – Conclusion

a. Observe Vanilla DiT's Dynamics with Hyperbolic head



9 HiT – Conclusion

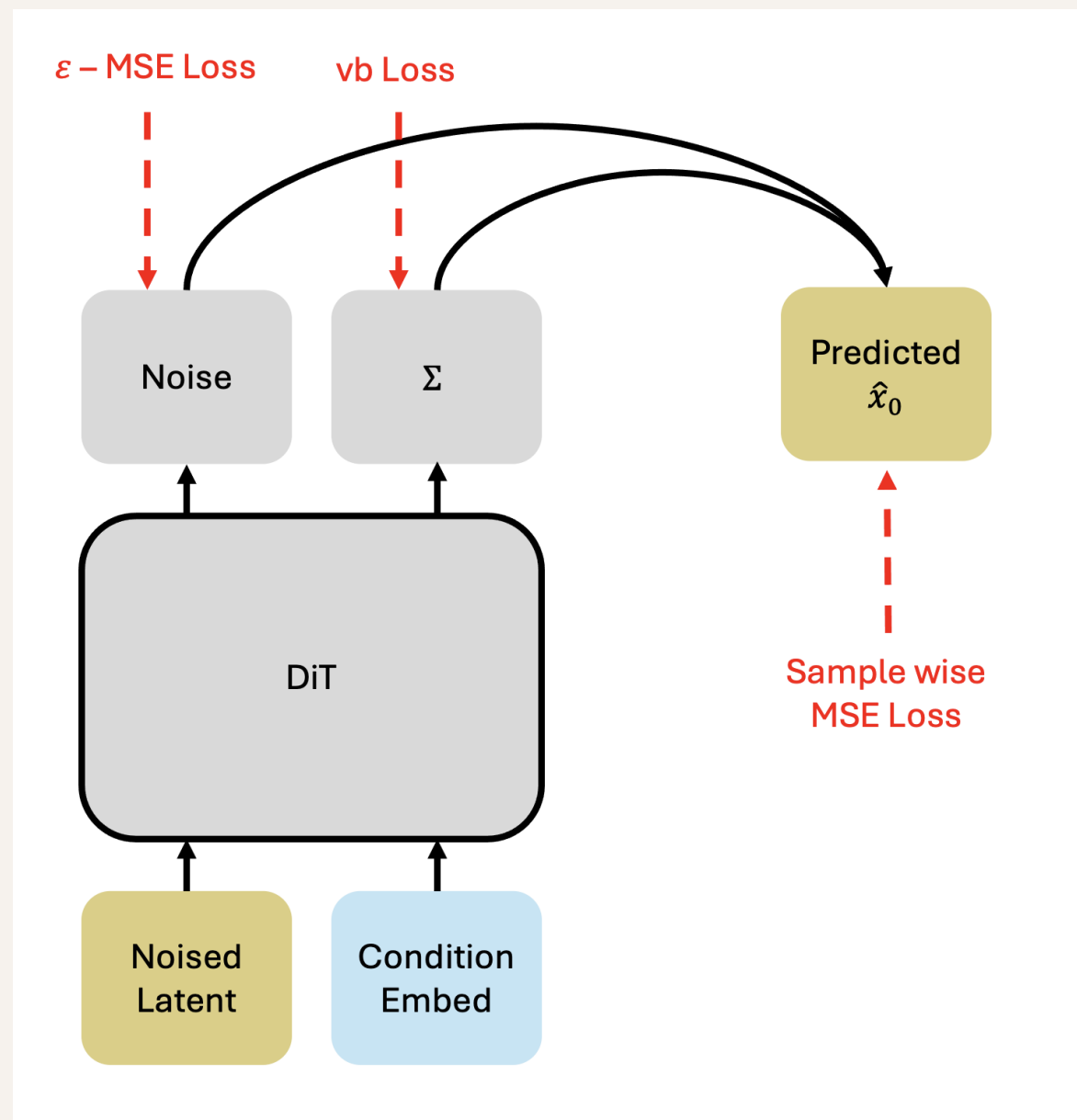
b. Light intervention & Ablation



# core	Model	# parameter	patch size	# steps	final train loss(MSE)	FID	note
2	DiT- b(a)	130M	2	100k	0.153	35.9	DiT standard loss
2	HiT- b(d)	130M	2	100k	0.147	21.89	standard loss + hyperbolic MSE

9 HiT – Conclusion

b. Light intervention & Ablation

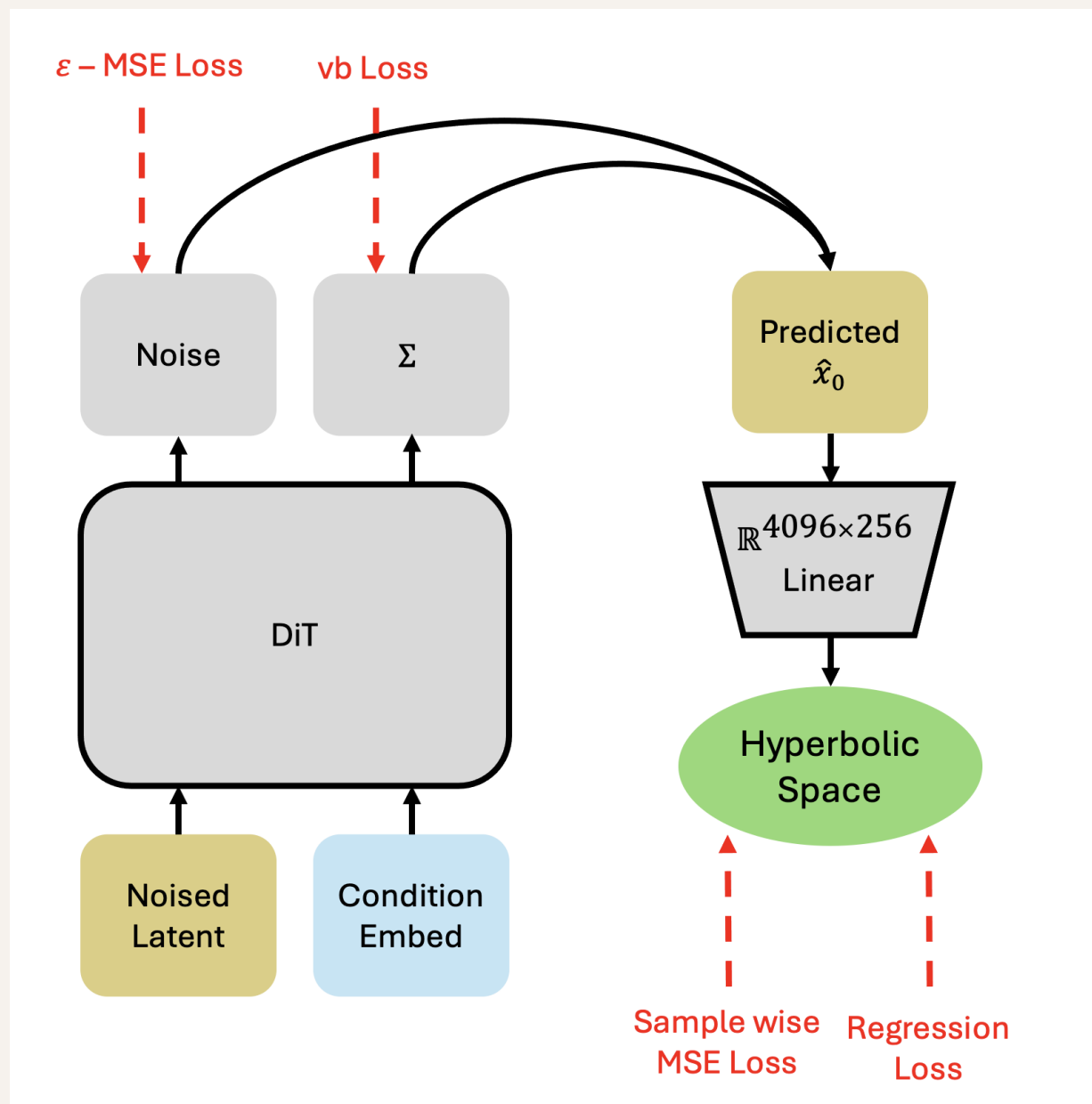


# core	Model	# parameter	patch size	# steps	final train loss(MSE)	FID	note
2	HiT-b(f)	130M	2	60k	0.151	44.52	standard loss + hyperbolic MSE
2	DiT-b(c)	130M	2	60k	0.149	35.5	causal-separation control

>> FID improve was intributed by more
Global Information from Sample-wise Loss
Not Hyperbolic space

9 HiT – Conclusion

c. Strong Intervention

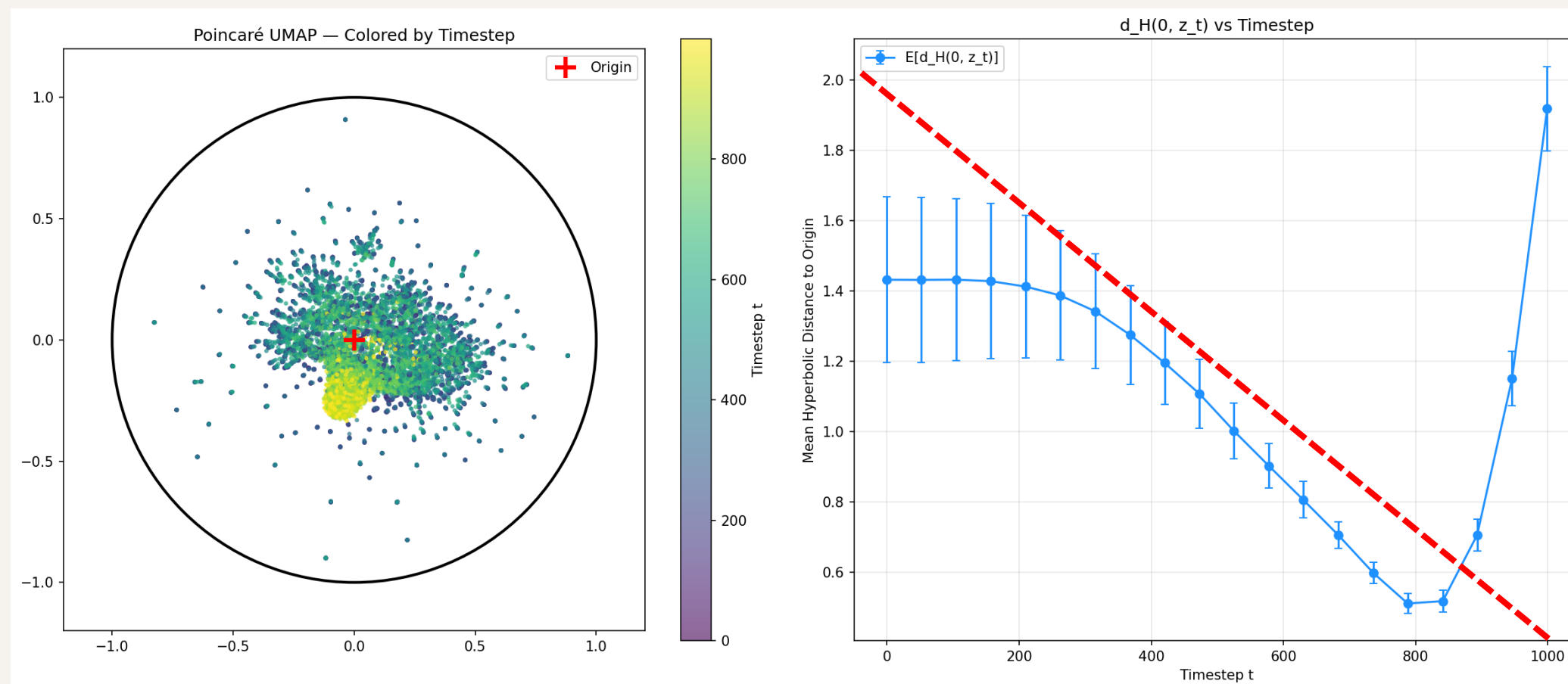


# core	Model	# parameter	patch size	# steps	regularization threshold t	final train loss(MSE)	FID	Frozen rho	note
2	HiT-b(i)	130M	2	60k	800	0.152	52.54	-0.08	inductive bias // linear reg toward manifold mean
2	HiT-b(j)	130M	2	60k	1000	0.154	85.29	-0.46	inductive bias // linear reg toward manifold mean
2	HiT-b(m)	130M	2	60k	800	0.152	50.55	-0.05	inductive bias // origin-aligned linear reg
2	HiT-b(n)	130M	2	60k	1000	0.154	90.92	-0.55	inductive bias // origin-aligned linear reg

>> The harder we forced the geometry, the more performance dropped

9 HiT – Conclusion

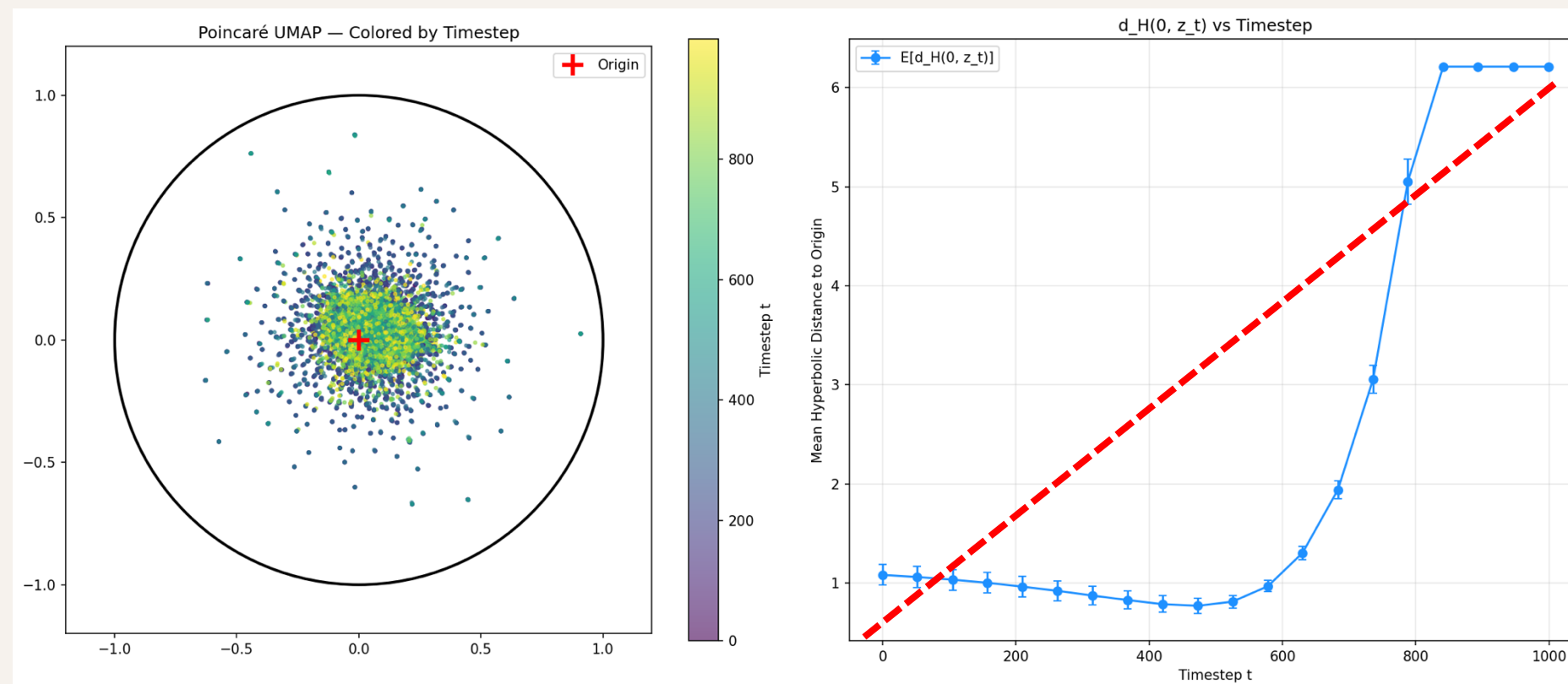
c. Strong Intervention



>> The harder we forced the geometry, the more performance dropped

9 HiT – Conclusion

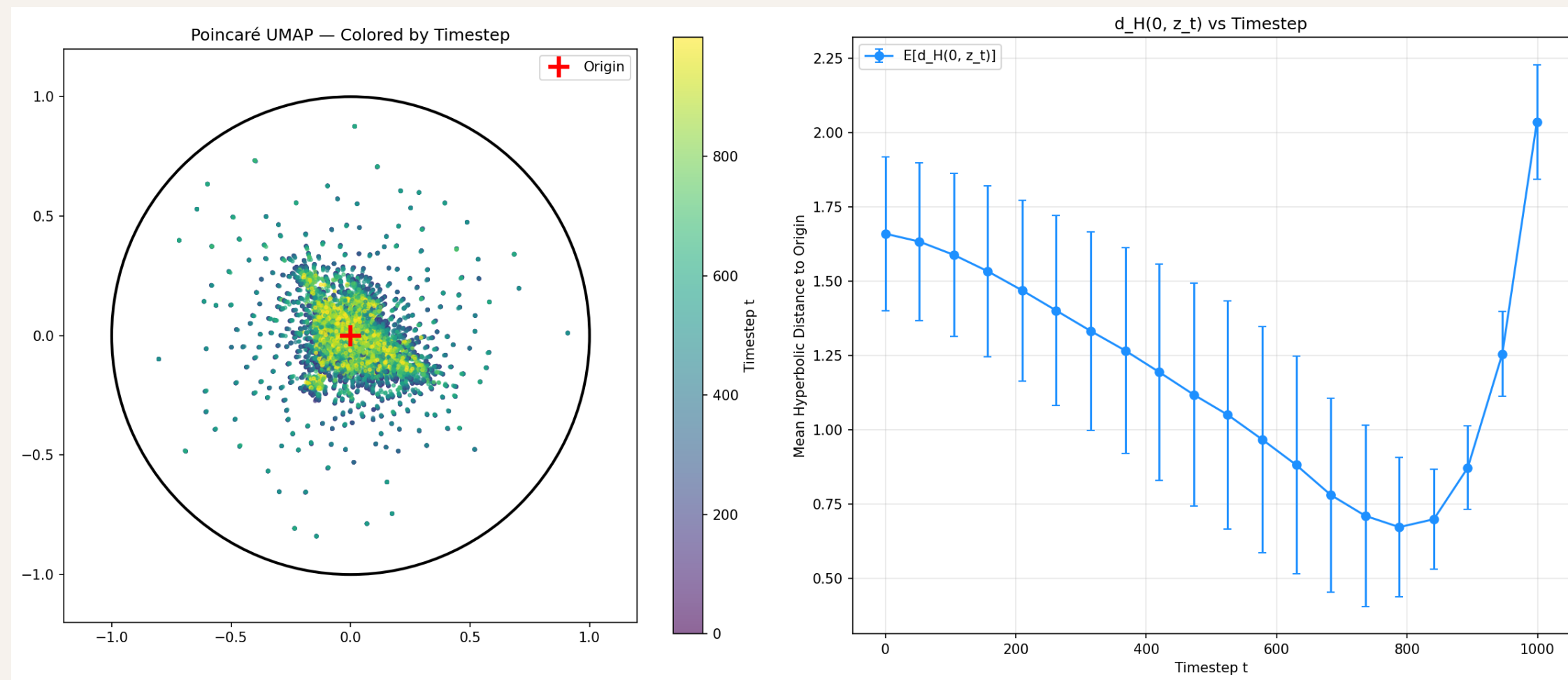
c. Strong Intervention – Opposite Hypothesis experiment



# core	Model	# parameter	patch size	# steps	regularization threshold t	final train loss(MSE)	FID	Frozen rho	note
2	HiT-b(g)	130M	2	60k	800	0.151	44.48	0.56	inductive bias // linear reg toward manifold mean, reversed
2	HiT-b(h)	130M	2	60k	1000	0.151	45.08	0.65	inductive bias // linear reg toward manifold mean, reversed

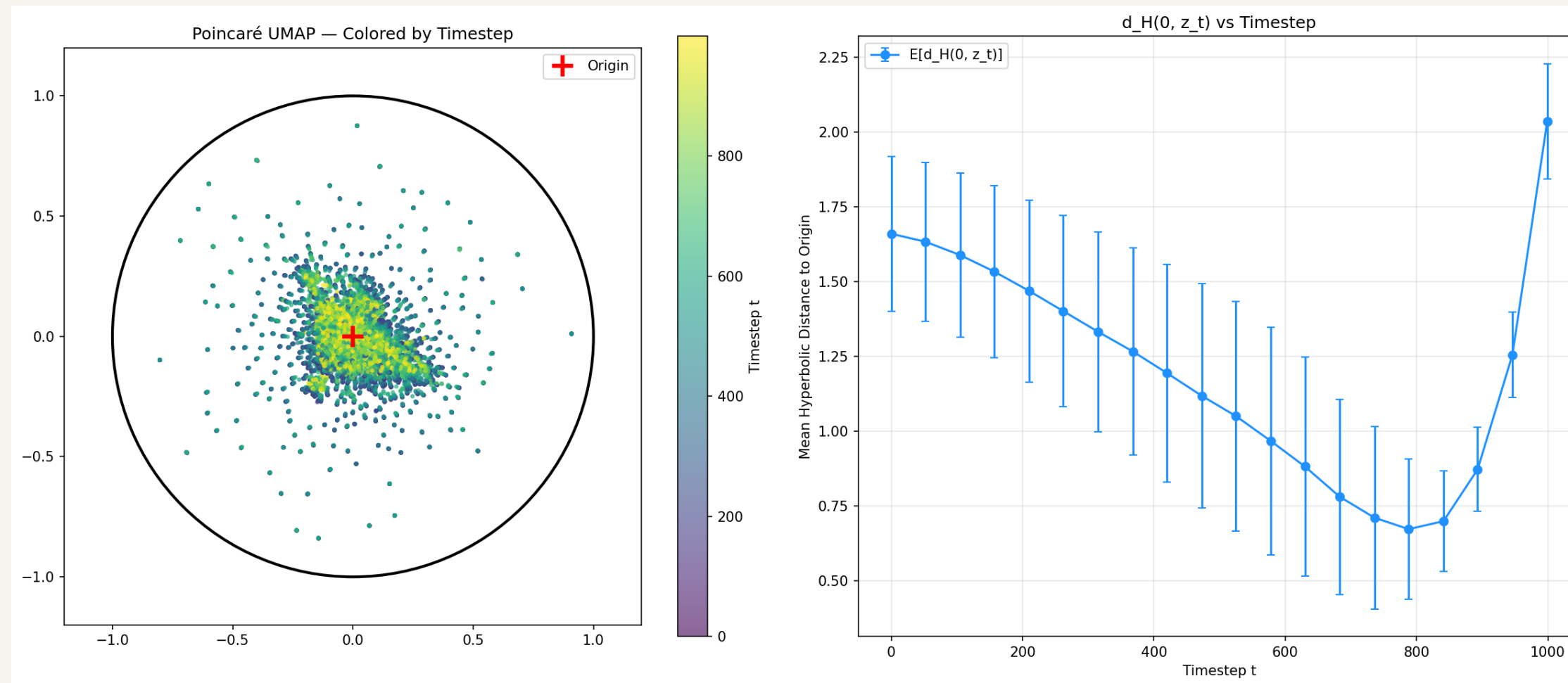
>> The monotonic descent up to 800 was not damaged; only the 800 to 1000 divergence accelerated. And the performance drop here was actually smaller than for the intuition-aligned regularizer. In other words, forcing the structure to match the intuition was the bigger loss.

10 HiT – Discussion



We discover U-curve Geometric in DiT's x_{pred} latent space
But, we failed to control.

10 HiT – Discussion



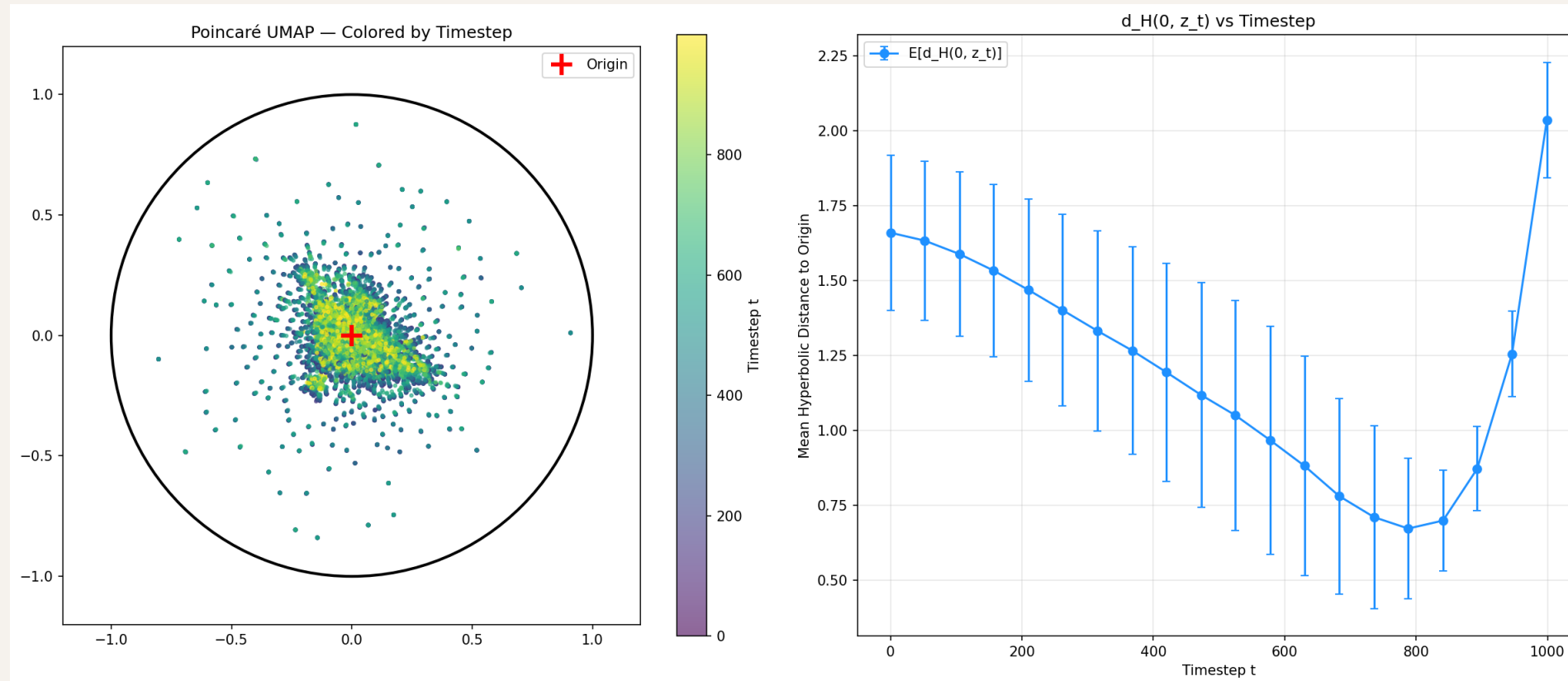
We discover U-curve Geometric in DiT's x_{pred} latent space

But, we failed to control.

>> So, was the hypothesis right?

10 HiT – Discussion

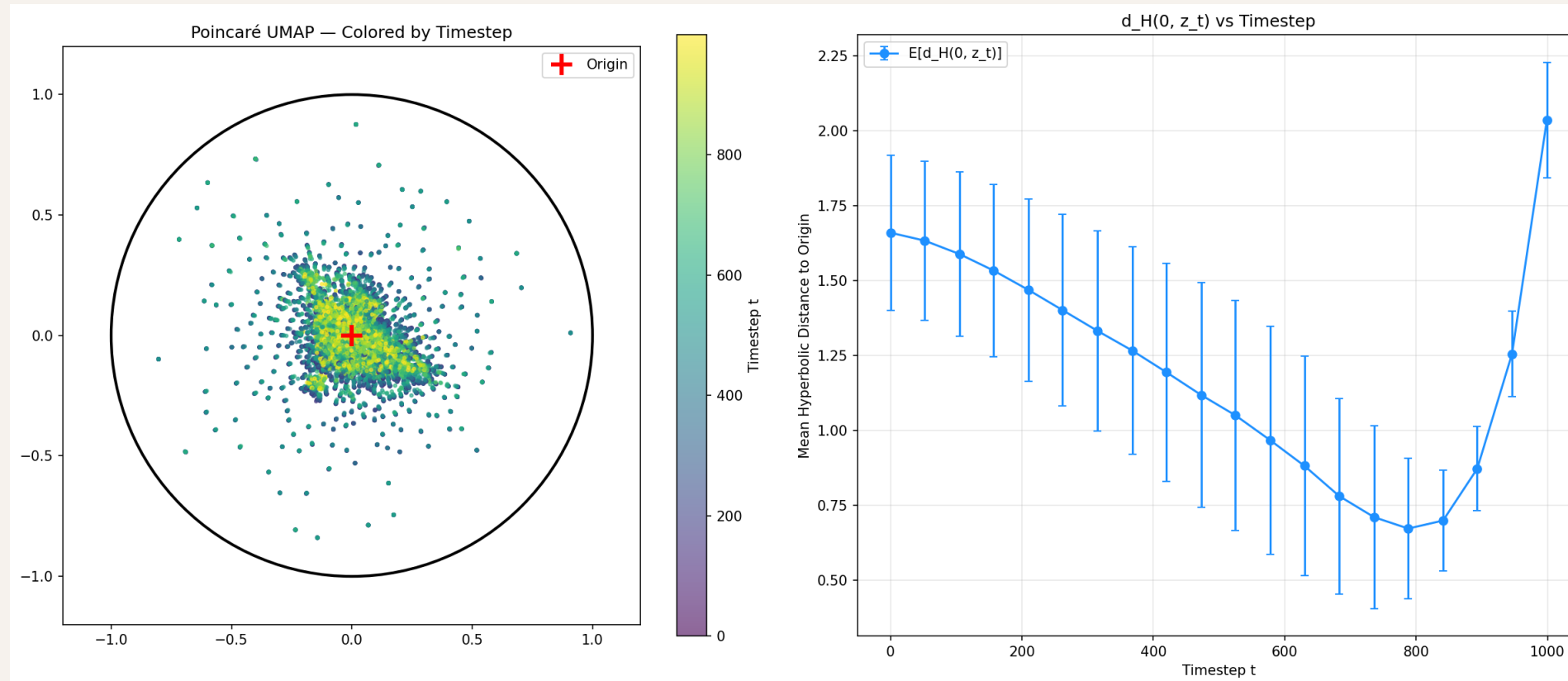
Another approach: with formular



$$x_t = \sqrt{(\bar{\alpha}_t)} \cdot x_0 + \sqrt{(1 - \bar{\alpha}_t)} \cdot \varepsilon,$$

10 HiT – Discussion

Another approach: with formular

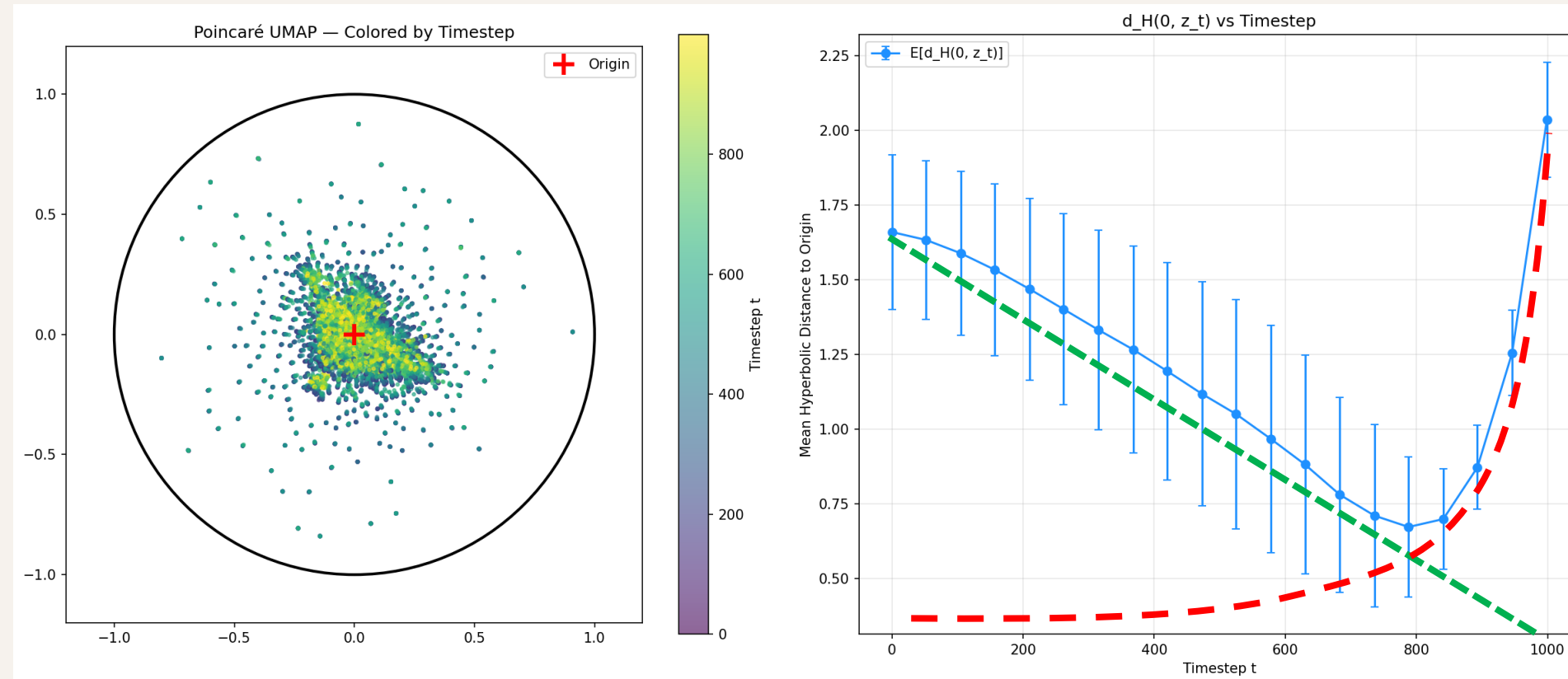


$$x_t = \sqrt{(\bar{\alpha}_t)} \cdot x_0 + \sqrt{(1 - \bar{\alpha}_t)} \cdot \varepsilon,$$

$$\hat{x}_0 = \frac{(x_t - \sqrt{(1 - \bar{\alpha}_t)} \cdot \varepsilon_\theta)}{\sqrt{(\bar{\alpha}_t)}}$$

10 HiT – Discussion

Another approach: with formular



$$x_t = \sqrt{(\bar{\alpha}_t)} \cdot x_0 + \sqrt{(1 - \bar{\alpha}_t)} \cdot \varepsilon,$$

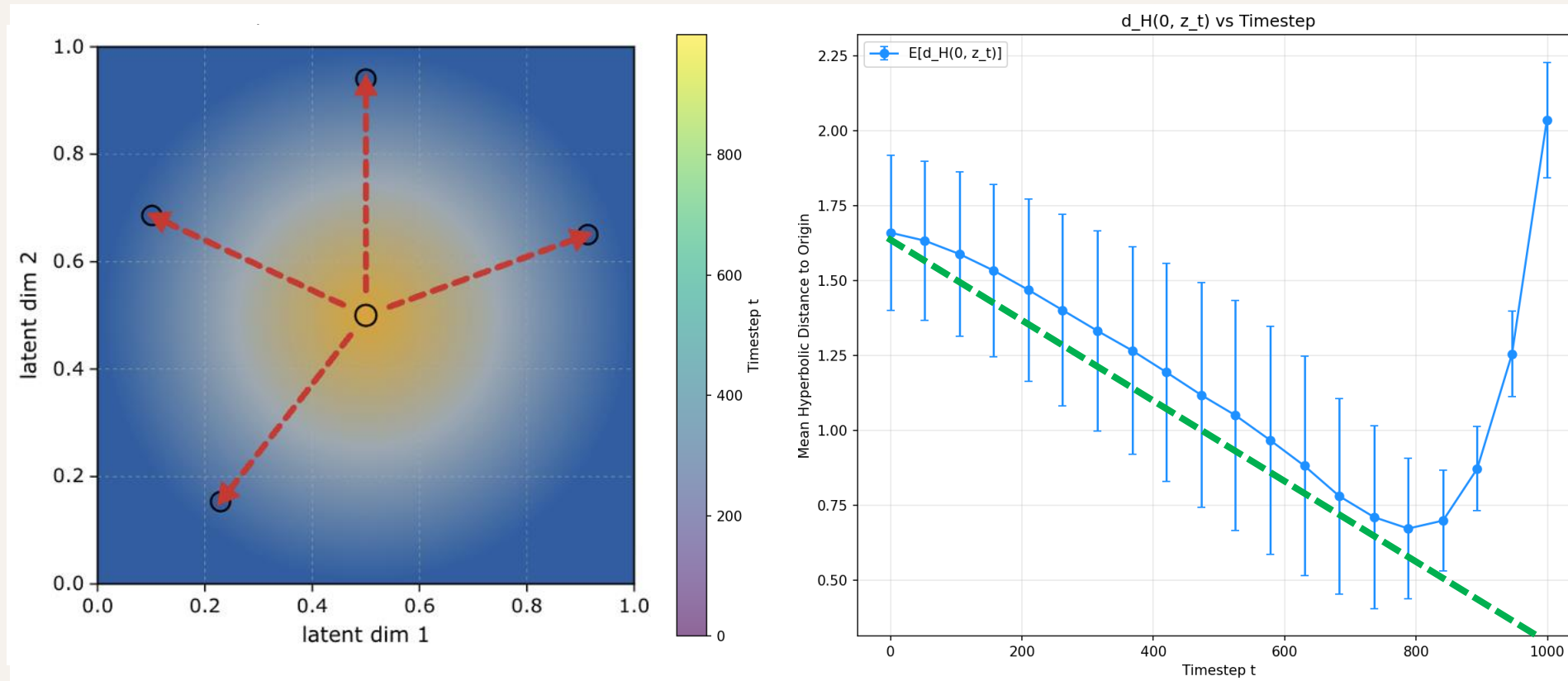
$$\hat{x}_0 = \frac{(x_t - \sqrt{(1 - \bar{\alpha}_t)} \cdot \varepsilon_\theta)}{\sqrt{(\bar{\alpha}_t)}}$$

$$\text{Let } \varepsilon_\theta = E[\varepsilon|x_t] + \delta$$

$$\hat{x}_0 = E[x_0|x_t] - k_t \cdot \delta, \quad (k_t = \frac{\sqrt{1 - \bar{\alpha}_t}}{\sqrt{\bar{\alpha}_t}})$$

10 HiT – Discussion

Another approach: with formular



$$x_t = \sqrt{(\bar{\alpha}_t)} \cdot x_0 + \sqrt{(1 - \bar{\alpha}_t)} \cdot \varepsilon,$$

$$\hat{x}_0 = \frac{(x_t - \sqrt{(1 - \bar{\alpha}_t)} \cdot \varepsilon_\theta)}{\sqrt{(\bar{\alpha}_t)}}$$

$$\text{Let } \varepsilon_\theta = E[\varepsilon|x_t] + \delta$$

$$\hat{x}_0 = E[x_0|x_t] - k_t \cdot \delta, \quad (k_t = \frac{\sqrt{1 - \bar{\alpha}_t}}{\sqrt{\bar{\alpha}_t}}) .$$

If model perfectly trained, $\delta = 0$, then same with our hypothesis

10 HiT – Discussion

Another approach: with formular

$$x_t = \sqrt{(\bar{\alpha}_t)} \cdot x_0 + \sqrt{(1 - \bar{\alpha}_t)} \cdot \varepsilon,$$

$$\hat{x}_0 = \frac{(x_t - \sqrt{(1 - \bar{\alpha}_t)} \cdot \varepsilon_\theta)}{\sqrt{(\bar{\alpha}_t)}}$$

$$\text{Let } \varepsilon_\theta = E[\varepsilon|x_t] + \delta$$

$$\hat{x}_0 = E[x_0|x_t] - k_t \cdot \delta, \quad (k_t = \frac{\sqrt{1 - \bar{\alpha}_t}}{\sqrt{\bar{\alpha}_t}}) .$$

If the model is perfectly trained, $\delta = 0$, then matches our hypothesis

Then Why control failed?

>> This is a **posterior Phenomenon**.

Control directly with regularization loss is like banning ice cream sales to reduce shark attacks.

References

Blayney, H., Arroyo, Á., Obando-Ceron, J., Castro, P. S., Courville, A., Bronstein, M. M., & Dong, X. (2026). A mechanistic analysis of looped reasoning language models. arXiv. <https://arxiv.org/abs/2604.11791>

Dieleman, S. (2024). Diffusion is spectral autoregression. <https://sander.ai/2024/09/02/spectral-autoregression.html>
Dosovitskiy, A., Beyer, L., Kolesnikov, A., Weissenborn, D., Zhai, X., Unterthiner, T., Dehghani, M., Minderer, M., Heigold, G., Gelly, S.,

Uszkoreit, J., & Houlsby, N. (2021). An image is worth 16x16 words: Transformers for image recognition at scale. In International Conference on Learning Representations (ICLR).

Khrulkov, V., Mirvakhabova, L., Ustinova, E., Oseledets, I., & Lempitsky, V. (2020). Hyperbolic image embeddings. In Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR) (pp. 6418–6428).

LeCun, Y., Maes, L., Le Lidec, S., Scieur, D., & Balestrieri, R. (2026). LeWorldModel. arXiv. <https://arxiv.org/abs/2603.19312>

Peebles, W., & Xie, S. (2023). Scalable diffusion models with transformers. In Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV) (pp. 4195–4205).

Ronneberger, O., Fischer, P., & Brox, T. (2015). U-Net: Convolutional networks for biomedical image segmentation. In Medical Image Computing and Computer-Assisted Intervention (MICCAI) (pp. 234–241). Springer.

Song, Y., & Ermon, S. (2019). Generative modeling by estimating gradients of the data distribution. In Advances in Neural Information Processing Systems (NeurIPS) (Vol. 32, pp. 11918–11930).

Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., Kaiser, Ł., & Polosukhin, I. (2017). Attention is all you need. In Advances in Neural Information Processing Systems (NeurIPS) (Vol. 30, pp. 5998–6008).



Thank you!

Q & A
